

BitFS 系统设计

本文档为 BitFS 设计文档体系的第二层：模块划分、接口定义、数据流。

文档体系：(总体设计) 1. 概念设计 — 项目愿景、核心概念、架构概览 2. 系统设计 (本文档) — 模块划分、接口定义、数据流 3. 详细设计 — 算法、数据结构、协议细节 4. 测试用例 — 测试用例设计

各节的详细设计（算法、数据结构、协议细节）见 [3-DetailedDesign.zh.md](#) 中对应的 B 节。

二、HD 钱包与 Vault

核心概念

重要: P_node 同时作为 Metanet 节点身份和加密操作的身份密钥。 - P_node / D_node: BIP32 HD 派生, 稳定, 用于 Metanet 节点标识、版本控制、ECDH 加密 - ECDH 加密: 直接使用 D_node 进行 ECDH, key_hash 在 KDF 阶段提供内容唯一性 (aes_key = KDF(ECDH(D_node, P_buyer), key_hash)) - key_hash = SHA256(SHA256(plaintext)): 双哈希值, 避免直接暴露原始内容哈希 (仍可被字典攻击匹配已知内容)

Vault: 一个独立的 Metanet 目录树。每个 Vault = 一个 BIP32 account = 一棵独立的 Metanet 树。

网络配置

网络绑定到种子级别: `bitfs init --network <network>` 时选定网络, 所有 Vault 共享。费用密钥链 (account 0) 与所有 Vault 在同一网络上。

预设网络:

网络	用途	地址前缀	P2P 端口	RPC 端口
mainnet	生产环境	0x00 (1...)	8333	8332
testnet	BSV 测试网 (STN)	0x6f (m/n...)	18333	18332
teratestnet	Teranode 测试网 (实验性)	0x6f	待定	待定

网络	用途	地址前缀	P2P 端口	RPC 端口
regtest	本地开发	0x6f	18444	18443

自定义网络: 通过 JSON 配置文件定义, 支持企业私链和新测试网:

```
{
  "name": "my-private-net",
  "address_version": "0x6f",
  "p2sh_version": "0xc4",
  "default_port": 19333,
  "rpc_port": 19443,
  "seeds": ["node1.company.internal:19333"],
  "genesis_hash": "0x00000000..."
}
```

多网络使用: 通过 BITFS_HOME 环境变量隔离:

```
BITFS_HOME=~/.bitfs-testnet bitfs init --network testnet
```

本地存储结构:

```
~/.bitfs/
├─ wallet.enc          # BITFS_HOME (可通过环境变量覆盖)
├─ ciphertext          # Argon2id 加密的 HD seed (salt || nonce ||
├─ config.toml         # 网络配置、默认 vault 等
├─ vaults/
│   └─ {vault_id}/     # 每个 vault 独立目录
│       ├── meta.json  # vault 元数据
│       └─ txstore/    # 本 vault 的交易 + Merkle proof
├─ cache/
│   ├── keys/          # 已购买文件的 AES 密钥缓存 (加密存储)
│   └─ meta/           # 元数据缓存
├─ spv/
│   ├── headers.db     # 区块头数据库
│   └─ peers.json      # P2P 节点列表
└─ store/              # 内容寻址存储 (本地文件内容)
```

密钥缓存加密: `cache/keys/` 中的密钥缓存文件使用 `wallet derived_key` 加密, 格式: `{nonce(12B) || AES-GCM(derived_key, nonce, key_data)}`。禁止以明文 JSON 存储 AES 对称密钥。

HD 钱包结构

```
HD Seed (BIP39 助记词 + 可选 passphrase 加密)
├── m/44'/236'/0' — 费用密钥链 (所有 Vault 共享)
│   ├── /0/M → 手续费/充值地址
│   └── /1/M → 找零地址
├── m/44'/236'/1' — Vault #0 (例: "personal")
│   ├── /0/0 → D_root / P_root 根目录 (DNSLink 身份)
│   ├── /0/0/1 → 子节点 #1 (树状派生, 镜像文件系统层次)
│   ├── /0/0/2 → 子节点 #2
│   └── /0/0/N/M → 深层嵌套子节点
├── m/44'/236'/2' — Vault #1 (例: "company")
│   ├── /0/0 → P_root_1 (独立根, 可绑独立 DNSLink)
│   └── ...
├── m/44'/236'/N' — Vault #N-1
└── 加密密钥: ECDH 直接使用 D_node (Method 42)
    aes_key = KDF(ECDH(D_node, P_recipient), key_hash)
    key_hash = SHA256(SHA256(plaintext))
    BIP32 非硬化派生的 ECDH 传递性 → 目录树级解密
```

HD 树状派生镜像文件系统层次: 根目录是 /0/0, 根的第一个子节点是 /0/0/1, 该子节点下的第一个子节点是 /0/0/1/1, 以此类推。

`file_index` 定义: Protobuf payload 的 `index` 字段 (field 14) - 节点创建时由父目录 `next_child_index` 分配的单调递增编号, 同时也是该节点 BIP32 HD 派生路径的最后一段。
`file_index` 决定 BIP32 派生的 `D_node`, `D_node` 直接用于 ECDH 加密。`ChildEntry` 的 `hardened` 字段决定该 `index` 使用硬化还是非硬化派生, 从而控制目录树购买时是否包含该子节点。

Vault 命令

```
bitfs vault create <name> # 创建新 Vault (创建 Metanet 根节点交易, 链上存储 Vault 元信息)
bitfs vault list          # 列出所有 Vault
bitfs vault use <name>    # 切换当前 Vault
bitfs vault info [name]   # 显示 Vault 详情
bitfs vault rename <old> <new> # 重命名 (更新链上元信息)
bitfs vault delete <name>  # 删除 Vault (标记删除)
```

Vault 元信息存储在根节点的 Metanet 交易中 (vault 名称、域名绑定等)。

钱包命令

```
bitfs wallet init      # 创建新 HD 钱包 (BIP39 助记词 + 可选 passphrase)
bitfs wallet restore   # 用助记词恢复 HD 密钥 (tx 数据需从备份恢复)
bitfs wallet info      # 余额、地址、网络
bitfs wallet fund      # 显示充值地址
```

wallet.enc 格式: 使用 Argon2id 从用户密码派生加密密钥, AES-256-GCM 加密 HD seed:

```
salt(16B) || nonce(12B) || AES-256-GCM(Argon2id(password, salt), nonce, seed ||
checksum)
```

- salt: 16 字节随机盐, 用于 Argon2id 密钥派生
- nonce: 12 字节随机数, 用于 AES-256-GCM
- checksum: seed 的 SHA256 前 4 字节, 用于验证解密正确性
- Argon2id 参数: m=64MB, t=3, p=4 (可调整)

初始化向导 (bitfs init)

```
bitfs init [--network <network>]
# 1. 选择网络 [mainnet / testnet / teratestnet / regtest / custom]
# 2. 生成 HD 钱包 (BIP39 助记词 + 可选 passphrase)
# 3. 提示备份助记词
# 4. 创建第一个 Vault
# 5. 显示充值地址
# 6. 链上发布根节点
```

充值工具 (可选 utility): - 系统内部全部使用 BSV - 提供 EVM 兼容链和 Solana 充值入口, 通过 DeFi 自动兑换为 BSV (低优先级)

详细设计: 派生规则、存储格式、恢复流程 → [二-B](#)

三、Unix 文件系统模型

BitFS 在链上实现了一个 Unix 文件系统。核心映射:

Unix 概念	BitFS 对应
inode	P_node (Metanet 节点公钥)
目录项 (dirent)	ChildEntry (index, name, type, pubkey)
文件名	仅存于父目录的 ChildEntry 中, 节点自身不存 name
· (当前目录)	节点自身 P_node
.. (父目录)	路径语义: 由遍历路径确定; parent 字段仅记录创建时父目录
硬链接	多个 ChildEntry → 同一 P_node (真正的 Unix 语义)
软链接 (symlink)	LINK_SOFT → 指向 P_node (最新版本)
远程软链接	LINK_SOFT_REMOTE → 指向 domain/path (跨用户)
parent 字段	首次创建父目录 (硬链接不改变)
inode 编号	P_node (节点身份, 硬链接共享同一 P_node)
dirent index	ChildEntry.index (父目录内序号, monotonic auto-increment)

三种节点类型

```

FILE  - 文件节点 (持有 key_hash, mime_type 等)
DIR   - 目录节点 (持有 children 列表, next_child_index)
LINK  - 链接节点 (仅用于软链接, 持有 link_target + link_type)

```

硬链接与软链接

硬链接 = 多个 ChildEntry 指向同一 P_node (真正的 Unix inode 语义)。不创建 LINK 节点, 仅是目录操作。

- 只允许文件硬链接, 禁止目录硬链接 (防止环路)
- 只能本 Vault 内 (类 Unix 不能跨文件系统)
- 删除: 仅删除 ChildEntry, 不维护引用计数, GC 留到后续工具

软链接 = LINK 节点:

类型	link_target	场景	解析
SOFT	P_node (33 bytes)	本 Vault 内	查最新 TxID (跟随更新), 支持链式跟随 (最大深度 10)
SOFT_REMOTE	domain/path (string)	跨 Vault/跨用户	DNSLink → Metanet 树遍历

`bitfs link` 默认硬链接, `bitfs link -s` 软链接。

操作语义

操作	实现	说明
<code>put</code> (新建)	分配新 HD index, op=CREATE	新节点, 更新父目录 ChildEntry
<code>put</code> (更新)	复用 P_node, op=UPDATE	同 P_node 新 TxID (Metanet 自动版本控制)
<code>rm</code>	2 笔交易: (1) SelfUpdate 父目录移除 ChildEntry, (2) 花费目标节点 UTXO 到 fee 地址	删除目录需先确保目录为空, 或使用 <code>rm -r</code> 递归删除
<code>mkdir</code>	分配新 HD index, type=DIR	新目录节点
<code>mv</code>	创建新节点 + 旧 P_node 变 SOFT 链接 → 新 P_node	新 HD 路径, 重新加密, 软链接自动跟随, 保持 HD 树镜像
<code>cp</code>	分配新 P_node, 重新加密 (新 file_index → 新 key)	真正复制 (独立新节点, 新 key_hash)
<code>link</code>	往目标目录添加 ChildEntry (复用 P_node)	硬链接 (不创建新节点, 仅目录操作)
<code>link -s</code>	分配新 P_node, type=LINK, SOFT	软链接 (创建 LINK 节点)

已知限制: 同一目录的写操作竞争同一 P_parent UTXO, 多客户端并发写入 会导致 UTXO 冲突 (双花)。缓解: daemon 实现目录级写锁 (乐观锁 + 自动重试)。

四、Metanet 交易格式

Metanet 协议核心要素

严格遵循 The Metanet Technical Summary: - Node = 一笔包含 OP_RETURN 的交易, 带
<Metanet Flag> <P_node> <TxID_parent> - Edge = 子交易的 Input 中包含 Sig P_parent
(父节点私钥签名, 花费锁定到 P_parent 的 UTXO) - Node ID = H(P_node || TxID_node) (全局
唯一) - 版本控制 = 同一 P_node 的多个 TxID, 区块高度/TOR 最高者为当前版本 - 权限控制 = 只
有 P_node 的私钥持有者才能创建子节点 (BSV 网络验证签名)

交易结构

核心设计: 元数据与内容完全分离。Metanet 节点交易仅存储元数据 (Protobuf payload), 内容
存储独立——默认链下 (daemon/LFCP 存储和服务), 链上可选 (单独的数据交易, OP_DROP 模
式)。

BitFS Metanet 节点交易 (元数据, 所有节点类型通用):

Inputs:

- Input 0: 花费锁定到 P_parent 的 UTXO
→ Sig(D_parent) 创建 Metanet Edge
- Input 1: 花费费用密钥链 UTXO (m/44'/236'/0')
→ 支付矿工费

Outputs:

- Output 0: OP_RETURN
 - ├─ <Metanet Flag> (4 bytes, "meta" = 0x6d657461)
 - ├─ <P_node> (33 bytes, 本节点压缩公钥)
 - ├─ <TxID_parent> (32 bytes, 父节点交易ID; 根节点为空)
 - └─ <BitFS Payload> (Protobuf 编码, 仅元数据/属性)
- Output 1: P2PKH → P_node (dust, 546 sat)
- Output 2: P2PKH → P_parent (dust, 546 sat) [必须]
- Output 3: P2PKH → 找零地址

BitFS 数据交易 (可选, 链上内容存储):

Inputs:

- Input 0: 花费任意 UTXO (Sig D_node 证明所有权)

Outputs:

- Output 0: <encrypted_content> OP_DROP
OP_DUP OP_HASH160 <H160(P_node)> OP_EQUALVERIFY OP_CHECKSIG
→ 加密内容在 spendable output, 锁定到 P_node
- Output 1: P2PKH → 找零

元数据/内容分离: Metanet 节点交易的结构始终相同, 无论内容存在链上还是链下。链下模式 (默认): daemon (LFCP) 存储和服务加密内容。链上模式 (可选): 额外发布一笔或多笔数据交易, 加密内容通过 OP_DROP 嵌入 spendable output。Protobuf 中的 onchain 标志和 content_txids 字段记录链上数据交易的引用。

UTXO 自持续链: 无需预先存入 BSV。每笔交易的 Output 2 刷新父节点 UTXO, 形成自持续链条。初始资金来自费用密钥链。

Protobuf Payload

```
syntax = "proto3";

enum Type { FILE = 0; DIR = 1; LINK = 2; }
enum Op { CREATE = 0; UPDATE = 1; DELETE = 2; }
enum Access { PRIVATE = 0; FREE = 1; PAID = 2; }
enum LinkType { SOFT = 0; SOFT_REMOTE = 1; }
enum CompressionScheme { COMPRESS_NONE = 0; COMPRESS_LZW = 1; COMPRESS_GZIP = 2; COMPRESS_ZSTD = 3; }

message ChildEntry {
  uint32 index = 1;    // 子节点在父目录中的编号
  string name = 2;     // 文件名/目录名 (名称仅存于此)
  Type type = 3;       // FILE / DIR / LINK
  bytes pubkey = 4;    // 子节点 P_node (33 bytes compressed)
  bool hardened = 5;   // true = 硬化派生 (目录购买排除, 需单独购买)
}

message RevShareEntry {
  bytes address = 1;   // 20-byte P2PKH hash (收益人地址)
  uint32 share = 2;    // 份额数量 (创作者定义总量, Covenant 守恒验证)
  string role = 3;     // 角色标签 (creator/editor/platform/...)
}

message ISOConfig {
  uint64 total_shares = 1; // 总发行量 (创作者决定)
  uint64 creator_reserve = 2; // 创作者自留份额
  uint64 offer_shares = 3; // 公开发售份额
  uint64 price_per_share = 4; // 每份价格 (satoshi)
  bytes iso_txid = 5; // ISO Genesis 交易 ID
}

message BitFSPayload {
  uint32 version = 1;    // 协议版本号
  Type type = 2;         // FILE / DIR / LINK
  Op op = 3;             // CREATE / UPDATE / DELETE

  // 文件属性 (FILE)
  string mime_type = 4;   // MIME 类型
}
```



```

// field 5 reserved (was encrypted_hash, 已移除 — 内容寻址由 key_hash 承担)
uint64 file_size = 6;          // 原始文件大小 (bytes)
bytes key_hash = 7;            // SHA256(SHA256(plaintext)) — 密钥派生 + 内容承诺 (双重哈希, 不暴露原始数据哈希)

// 访问控制
Access access = 8;             // PRIVATE / FREE / PAID
uint64 price_per_kb = 9;       // 单价: satoshis/KB (仅 PAID, 支持继承)

// 链接 (LINK, 仅软链接使用)
bytes link_target = 10;         // SOFT: P_node / SOFT_REMOTE: domain/path
LinkType link_type = 11;       // 链接类型 (SOFT / SOFT_REMOTE)

// 时间与导航
uint64 timestamp = 12;         // 操作时间 (Unix)
bytes parent = 13;             // 首次创建父目录 P_node (根节点指向自身, 硬链接不改变)
uint32 index = 14;            // 本节点在父目录中的 index

// 目录 (DIR)
repeated ChildEntry children = 15; // 子节点列表
uint32 next_child_index = 16;    // 下一个可用子节点编号 (monotonic auto-increment)

// 发布
string domain = 17;            // DIR: 绑定的域名 (双向 DNSLink 验证)

// 元信息
string keywords = 18;          // 空格分隔的关键词
string description = 19;       // 简短描述
map<string, string> metadata = 20; // 自定义键值对 (灵活扩展)

// 版本控制 (低优先级)
bytes version_log = 21;         // 指向版本记录 Metanet 节点的 P_node

// 共享 (远期: 群签名阶段)
bytes share_list = 22;         // 指向共享列表 Metanet 节点的 P_node

// PRIVATE 模式支持 (明文 envelope, 供钱包恢复)
bool encrypted = 23;           // true = 加密模式 (其余字段为默认值)
bytes private_key_hash = 24;    // key_hash 明文副本 (供恢复 aes_key = KDF(ECDH(D_node, P_node), key_hash))
bytes enc_payload = 25;        // nonce(12B) || 加密后的完整 Protobuf || GCM_tag(16B)
uint32 private_file_index = 26; // file_index 明文副本 (供恢复 D_node 的 HD 路径, 与 private_key_hash 配合)

// === 新增字段 (专利 US 2021/0399898 A1 对齐) ===

// 内容存储模式
bool onchain = 27;             // true = 内容已发布到链上数据交易
repeated bytes content_txids = 28; // 链上数据交易 TxID 列表 (onchain=true 时)

// 数据压缩

```

```

CompressionScheme compression = 29;    // 压缩方案

// 内容分片 (链上大文件)
uint32 chunk_index = 30;                // 本 chunk 序号 (0-based)
uint32 total_chunks = 31;               // 总 chunk 数 (0 = 非分片)
bytes recombination_hash = 32;           // H(chunk0 || chunk1 || ...) 完整性校验

// Rabin 签名 (内容认证)
bytes rabin_signature = 33;              // Rabin 签名 (S, U) 序列化
bytes rabin_pubkey = 34;                 // Rabin 公钥 n

// 区块高度权限
uint32 cltv_height = 35;                 // CLTV 区块高度 (0 = 无限制)

// 收益权表 (Revenue Share)
bytes registry_txid = 36; // 指向 Registry UTXO 所在交易
uint32 registry_vout = 37; // Registry UTXO 的输出索引
ISOConfig iso = 38; // ISO 配置 (可选, 仅 ISO 发起时写入)

// 网络标识 (仅根节点, 信息性)
string network_name = 39; // "mainnet" / "testnet" / "teratestnet" / "regtest" / 自定义

// ACL 引用
bytes acl_ref = 40; // ACL 引用 (群签名公钥哈希或 ACL 规则 TxID)

// 收益分成
uint32 revenue_share = 41; // 收益分成比例 (0-10000, 表示 0.00%-100.00%)
}

```

已知限制: 目录 children 列表采用 repeated ChildEntry, 每次添加/删除子节点 需重写完整列表。对于包含数千文件的大目录, SelfUpdate 交易体积和费用线性增长。缓解: (1) 建议单目录不超过 1000 个子节点; (2) 远期可探索增量更新或 Merkle Trie。

注: 加密密钥直接通过 ECDH(D_node, P_recipient) 派生, key_hash 在 KDF 阶段提供内容唯一性: `aes_key = KDF(ECDH(D_node, P_recipient), key_hash)`。这保留了 BIP32 的代数关系, 使目录树购买成为可能。

field 5 说明: 原 `encrypted_hash` 字段已移除。内容寻址和密钥派生统一由 `key_hash = SHA256(SHA256(plaintext))` 承担双重职责: (1) KDF 密钥派生的 salt 参数, (2) 内容完整性承诺 (下载后验证)。链下内容寻址 (daemon 内部存储索引) 是实现细节, 不需要在 Protobuf 协议层暴露。

三种数据类型的交易差异

加密体系: Koblitz (secp256k1 ECC) 加密对称密钥 + AES-256-GCM 加密内容 (详见第五节)。

- FREE: $ECDH(D_node=1, P_node)$ trick $\rightarrow S_k = KDF(ECDH(1, P_node), key_hash)$ 可公开计算, $AES-GCM(content, S_k)$, P_node 通过 DNSLink 公开
- PAID: $aes_key = KDF(ECDH(D_node, P_node), key_hash)$ — 与 PRIVATE 同密钥基础, 买家通过 HTLC/Token 获取 capsule $\rightarrow$ 还原 aes_key , $AES-GCM(content, aes_key)$; CDN 带宽费另行通过 x402 收取
- PRIVATE: $ECDH(D_node, P_node) \rightarrow$ 仅 Owner 可解密, Protobuf 内部加密
($encrypted=true$, $private_key_hash + private_file_index$ 明文, $enc_payload$ 加密)
 - $private_key_hash + private_file_index$ 在明文中供钱包恢复: $aes_key = KDF(ECDH(D_node, P_node), key_hash)$
 - 解密后的 $enc_payload$ 包含完整 Protobuf (mime_type, file_size, timestamp 等)

价格继承

`price_per_kb` 支持从父节点继承: 文件 $\rightarrow$ 父目录 $\rightarrow \dots \rightarrow$ 根节点。查找到第一个设置了该字段的节点即停止。

```
/ (root)           price_per_kb=100
├ docs/            price_per_kb=50
│   ├── readme.txt (继承 docs  $\rightarrow$  50 sat/KB)
│   └ guide.pdf    price_per_kb=200 (覆盖)
└ free/            access=FREE
```

详细设计: 交易模板、14 种操作、OP_RETURN 格式 $\rightarrow$ [四-B](#)

五、数据类型与加密模型

三种数据类型

类型	DNSLink	P_node	元数据	内容	解密密钥
私有	无	不公开	Protobuf 内部加密 ($encrypted=true$)	加密	仅 Owner ($D_node \rightarrow S_k$)

类型	DNSLink	P_node	元数据	内容	解密密钥
公开免费	有	公开	明文	加密 (D_node=1)	任何人: aes_key = KDF(ECDH(1, P_node), key_hash)
公开付费	有	公开	明文(含价格)	加密	购买后通过 Token/HTLC 获取 S_k

加密体系 (Koblitz + AES-256-GCM 混合, 与 Bitcoin 同密码体系)

所有文件都加密存储。采用 Koblitz 椭圆曲线加密 (secp256k1 ECC) 保护对称密钥, AES-256-GCM 加密批量内容:

密钥加密 (Koblitz, secp256k1 ECC):

对对称密钥 S_k (32 bytes) 逐字节加密:

1. 映射到曲线点: $P_m = m \times G$ (m = 字节值)
2. 随机临时密钥 k : 密文 = $(k \times G, P_m + k \times P_{\text{recipient}})$
3. 解密: $P_m = (P_m + k \times P_{\text{recipient}}) - D_{\text{recipient}} \times (k \times G)$, 恢复 m

适用: 加密 32 字节的 S_k → 约 2KB 密文 (可接受)

内容加密 (AES-256-GCM):

1. 生成随机对称密钥 S_k (32 bytes)
2. $\text{nonce} = \text{random}(12 \text{ bytes})$
3. $\text{encrypted} = \text{nonce} || \text{AES-256-GCM}(\text{content}, S_k, \text{nonce}) || \text{tag}$

适用: 批量内容加密 (高效)

完整加密流程:

1. 双哈希: $\text{key_hash} = \text{SHA256}(\text{SHA256}(\text{plaintext}))$
2. ECDH 直接使用 D_node: $\text{point} = \text{ECDH}(D_{\text{node}}, P_{\text{recipient}})$
3. 对称密钥: $\text{aes_key} = \text{KDF}(\text{point}, \text{key_hash})$
 $\text{KDF} = \text{HKDF-SHA256}(\text{ikm}=\text{point.x}, \text{salt}=\text{key_hash}, \text{info}=\text{"bitfs-method42"})$
4. Koblitz 加密 aes_key: $\text{koblitz_envelope} = \text{Koblitz_Encrypt}(\text{aes_key}, P_{\text{recipient}})$
5. AES 加密内容: $\text{encrypted_content} = \text{nonce} || \text{AES-256-GCM}(\text{content}, \text{aes_key}) || \text{tag}$
6. 存储:
 - Metanet 交易: key_hash (双哈希) 在 Protobuf 中
 - 链下: daemon 存储 encrypted_content
 - 链上 (可选): 发布数据交易, $\text{Output} = \text{koblitz_envelope} || \text{encrypted_content}$

解密流程 (Owner):

1. 从 Metanet 交易获取 key_hash
2. ECDH: $\text{point} = \text{ECDH}(D_{\text{node}}, P_{\text{node}}) = D_{\text{node}} \times P_{\text{node}}$
3. $\text{aes_key} = \text{KDF}(\text{point}, \text{key_hash})$
4. AES-256-GCM 解密内容
5. 验证: $\text{SHA256}(\text{SHA256}(\text{decrypted})) == \text{key_hash}$

解密流程（买家）：

1. 通过 Token/HTLC 获取 capsule (ECDH shared secret)
2. $\text{aes_key} = \text{KDF}(\text{capsule}, \text{key_hash})$
3. AES-256-GCM 解密内容
4. 验证: $\text{SHA256}(\text{SHA256}(\text{decrypted})) == \text{key_hash}$

加密密钥派生（调整后）：

ECDH 直接使用 D_node (BIP32 节点密钥), key_hash 移到 KDF 阶段：

旧: $\text{aes_key} = \text{KDF}(\text{ECDH}(\text{Df}(0), \text{P_buyer}))$ ← SHA256 打断 BIP32 关系
新: $\text{aes_key} = \text{KDF}(\text{ECDH}(\text{D_node}, \text{P_buyer}), \text{key_hash})$ ← 保留 BIP32 代数关系

这一调整使得 BIP32 非硬化派生的传递性可用于目录树级解密: - $\text{S_child} = \text{S_parent} + \text{offset} \times \text{P_buyer}$ (offset 从 xpub 的 chain code 派生) - key_hash 在 KDF 阶段提供内容唯一性, 不影响 ECDH 的 BIP32 可推导性

- key_hash 双重用途: $\text{key_hash} = \text{SHA256}(\text{SHA256}(\text{plaintext}))$ 同时用于 (1) KDF 密钥派生的 salt 参数, 和 (2) 内容完整性承诺 (下载后验证)。不再需要单独的 encrypted_hash 字段 — 链下内容寻址是 daemon 内部实现细节。
- 免费数据: D_node=1 技巧 — $\text{aes_key} = \text{KDF}(\text{ECDH}(1, \text{P_node}), \text{key_hash}) = \text{KDF}(\text{P_node}, \text{key_hash})$, P_node 通过 DNSLink 公开 → 任何人可计算
- 私有/付费数据: ECDH 直接使用 D_node — $\text{aes_key} = \text{KDF}(\text{ECDH}(\text{D_node}, \text{P_node}), \text{key_hash})$

私有数据的隐私保护

- 不设置 DNSLink, 不公开 P_node
- Protobuf 内部加密: encrypted=true, private_key_hash + private_file_index 明文 (供钱包恢复), enc_payload 加密
- 链上可见: version + encrypted 标记 + private_key_hash (双哈希) + private_file_index + 加密 blob
- 隐私边界 (公链固有权衡):
 - 数据内容: 不可见 (加密)
 - 路径/文件名/元数据: 不可见 (enc_payload 内)
 - 图结构: 可见 — Metanet 协议要求 P_node 和 TxID_parent 在 OP_RETURN 中明文, 因此父子关系、操作频率、树结构深度等模式仍可被观察
 - 内容指纹: 可关联 — private_key_hash 是确定性的, 攻击者可通过字典攻击匹配已知内容

◦ 缓解策略:

- 威胁边界: 仅对”已知内容确认”有效 (如验证某用户是否存储了特定已知文件), 无法反推未知内容
- key_hash 使用双哈希 (SHA256(SHA256(plaintext))), 增加一次哈希的暴力搜索成本
- 对高敏感场景, Owner 可在加密前对 plaintext 添加随机 padding (应用层处理, 协议不强制)
- 长远方案: 未来版本可引入 per-file salt 混入 key_hash 计算, 代价是需要额外字段存储 salt

六、DNSLink、Paymail 与发布

DNS 记录

域名绑定使用两种 DNS 记录:

```
;; 身份: P_node 公钥 (TXT, 唯一)
_bitfs_pubkey.example.com    TXT    "02a1b2c3d4e5f6..."

;; 服务端点: SRV 记录 (支持多个, CDN 负载均衡)
_bitfs_tcp.example.com      SRV    10 60 443 cdn1.example.com
_bitfs_tcp.example.com      SRV    10 40 443 cdn2.example.com
_bitfs_tcp.example.com      SRV    20 100 443 backup.example.com
```

_bitfs_pubkey (TXT): P_node 公钥 (33 bytes 压缩公钥的 hex 编码)。只能有一个。P_node 可以是任意 Metanet 节点 - 不限于 Vault 根目录, 可以是任何目录甚至文件。

_bitfs_tcp (SRV): 数据服务端点。SRV 记录格式: `priority weight port target`。 - `priority`: 数字越小优先级越高, 客户端优先连接低 priority 的服务器 - `weight`: 同优先级内的负载均衡权重 (按比例分配流量) - `port`: 服务端口 (通常 443) - `target`: 服务器域名或 IP - 支持多个记录 → 天然 CDN 负载均衡, 完全由 DNS 控制

双向验证

双向验证增强安全性: 1. DNS → Metanet: `_bitfs_pubkey` TXT 记录指向 P_node 2. Metanet → DNS: 目录 payload 中的 domain 字段记录绑定的域名

两个方向必须一致。防止 DNS 被篡改后指向恶意节点。

默认文件

P_node 指向目录时, 访问 `bitfs://example.com/` 默认返回目录下的 `index.html` 文件 (类似 Web 服务器)。P_node 指向文件时, 直接返回该文件内容。

Publish 支持任意目录

`publish` 不限于根目录, 任何 Metanet 节点 (目录或文件) 都可以绑定域名:

```
bitfs publish <domain> [path]    # 绑定域名到指定目录 (默认 /)
bitfs unpublish <domain>         # 解除绑定
bitfs publish                     # 查看所有绑定关系
```

寻址方式

- `bitfs://domain.com/path` - 通过 DNSLink 解析
- `bitfs://<pubkey>/path` - 直接用公钥 (无需 DNS)

解析链路:

```
bls bitfs://example.com/docs/
  → DNS TXT lookup _bitfs_pubkey.example.com → 得到 P_node
  → DNS SRV lookup _bitfs_tcp.example.com → 得到 endpoint(s) (按 priority/weight 排序)
  → 连接最优 endpoint, Method 42 握手验证身份
  → 从 daemon 获取 Metanet 元数据 (SPV: 不查链)
  → 验证 domain 双向一致
  → 遍历目录树 → 返回结果

bitfs://example.com/ (P_node 指向目录)
  → 同上解析 → 返回 index.html 的内容

bitfs://example.com/ (P_node 指向文件)
  → 同上解析 → 直接返回文件内容
```

Paymail 集成

BitFS 集成 [Paymail \(bsvalias\)](#) 作为补充身份层, 实现 email 风格的人类友好寻址。

扩展 URI 格式

```
bitfs://alice@example.com/docs/paper.pdf
      ↑           ↑
      Paymail alias domain
```

三种寻址方式并存:

格式	示例	解析方式
Paymail	<code>bitfs:// alice@example.com/ path</code>	@ → Paymail 协议
DNSLink	<code>bitfs://example.com/ path</code>	无 @, 非 hex → <code>_bitfs_pubkey</code> TXT
裸公钥	<code>bitfs://02a1b2c3.../ path</code>	无 @, hex 开头 → 直连

URI 解析优先级:

```
parse(uri):  
    authority = uri.authority  
    if '@' in authority:  
        alias, domain = split(authority, '@')  
        → Paymail: SRV _bsvalias._tcp.{domain} → .well-known/bsvalias → capabilities  
    elif is_hex_pubkey(authority):  
        → 直连: 公钥即 P_node, 需额外提供 endpoint  
    else:  
        → DNSLink: TXT _bitfs_pubkey.{domain} + SRV _bitfs._tcp.{domain}
```

Paymail 优势: 一域多用户

DNSLink 模式一个域名只能绑定一个 P_node。Paymail 实现多用户:

```
alice@example.com → Vault A 的 P_root  
bob@example.com   → Vault B 的 P_root
```

同一 daemon 实例可托管多个 Vault, 通过 Paymail alias 区分。

Daemon 作为 Paymail Server

BitFS daemon 同时暴露 Paymail capabilities:

```
GET https://example.com/.well-known/bsvalias  
{  
  "bsvalias": "1.0",  
  "capabilities": {  
    "pki": "https://example.com/api/v1/pki/{alias}@{domain.tld}",  
    "f12f968c92d6": "https://example.com/api/v1/profile/{alias}@{domain.tld}",  
    "a9f510c16bde": "https://example.com/api/v1/verify/{alias}@{domain.tld}/{pubkey}"  
  }  
}
```



```
}
}
```

Capability	BRFC ID	映射
PKI	pki	→ Vault 的 P_root 公钥
Public Profile	f12f968c92d6	→ Vault metadata (name, description, avatar)
Verify Public Key	a9f510c16bde	→ 验证公钥是否属于该 Vault

DNS 记录

Paymail 与 DNSLink 共存, 指向同一服务器:

```
;; DNSLink (现有, 保留)
_bitfs_pubkey.example.com    TXT    "02a1b2c3d4e5f6..."
_bitfs_tcp.example.com       SRV     10 60 443 cdn1.example.com

;; Paymail (新增, 同一 host)
_bsvalias._tcp.example.com   SRV     10 60 443 cdn1.example.com
```

Paymail 在各场景的应用

场景	现状	Paymail 后
文件寻址	bitfs:// example.com/path	bitfs://alice@example.com/path
买方身份	P_buyer (33字节公钥)	bob@handcash.io (握手时出示)
销售记录	Buyer: 02a1b2c3...	Buyer: bob@handcash.io
远程软链接	bitfs link -s example.com/path	bitfs link -s alice@example.com:/ path
Method 42 握手	原始公钥互验	Paymail PKI 查找公钥 + 验证
收益权股东	BSV address	Paymail → 每次分红获取新地址 (HD 派生, 增强隐私)
Shell 交互		open alice@example.com

场景	现状	Paymail 后
	open bitfs:// example.com/	

不使用 Paymail 的场景

链上协议仍用原始公钥, Paymail 仅为链下身份发现: - Metanet P_node (链上) - HTLC 脚本中的地址 (链上) - Protobuf payload 字段 (链上) - ECDH 密钥派生 (密码学操作)

Go 库

使用 github.com/bsv-blockchain/go-paymail (BSV 官方 Go 实现): - 完整 client + server 支持 - 支持 mainnet/testnet/STN - BRFC 管理、SRV 解析、PKI、P2P 支付 - 与 BitFS 现有 Go 技术栈匹配

七、SPV 模式

全局原则: 所有交易信息 + Merkle proof 本地保存, 从不检索区块链。

数据来源

角色	数据获取方式
Owner	自己创建的交易 → 本地保存 tx + Merkle proof
Visitor	从 Owner 的 daemon 获取元数据 (主要方式)
降级	第三方索引服务作为备用 (daemon 不可达时)

SPV 验证闭环

Visitor 从任何来源获取数据后, 必须完成以下校验链才能信任数据:

1. 交易完整性: 获取完整交易 (tx bytes), 验证 OP_RETURN 中 Protobuf 可正确反序列化
2. Merkle 证明: 获取 Merkle proof (tx hash → Merkle root), 逐层验证哈希路径
3. 区块头验证: 对应 block header 的 Merkle root 与步骤 2 一致
4. 最长链检查: block header 属于已知的最长链 (通过 header chain 或检查点验证)
5. 内容完整性: 下载解密后, $\text{SHA256}(\text{SHA256}(\text{plaintext})) == \text{key_hash}$ (Protobuf 中的承诺)

失败策略: 任一步骤失败 → 拒绝数据, 标记来源不可信, 尝试降级到其他数据源。

本地存储

本地目录结构参见第二节「本地存储结构」(~/bitfs/ 统一定义)。SPV 相关数据存放于：

- ~/bitfs/spv/headers.db — 区块头数据库 (SPV 轻节点)
- ~/bitfs/spv/peers.json — P2P 节点列表
- ~/bitfs/vaults/{vault_id}/txstore/ — 各 vault 的交易 + Merkle proof
- ~/bitfs/cache/meta/ — Metanet 元数据缓存
- ~/bitfs/cache/keys/ — 已购买文件的 AES 密钥缓存 (加密存储)

恢复

`bitfs wallet restore` 恢复 HD 密钥，tx 数据需从用户备份恢复 (~/bitfs/ 目录)。系统不包含自动 tx 备份机制。

八、b* 独立工具 (只读/无状态/Visitor)

统一参数模式: `b<cmd> [OPTIONS] bitfs://<alias@domain|domain|pubkey>/<path>`

bls – 列目录 (类 Unix ls)

```
$ bls bitfs://example.com/docs/
readme.txt 4.2 KB 2026-02-14 [free]
images/    dir    2026-02-13
report.pdf 1.2 MB 2026-02-10 [paid: 50 sat/KB]

$ bls -l bitfs://example.com/docs/ # 详细模式
$ bls -json bitfs://example.com/   # JSON 输出 (Agent 友好)
$ bls -keyword "finance" bitfs://example.com/ # 按关键词过滤
$ bls bitfs://alice@example.com/docs/ # Paymail 寻址
```

bstat – 节点元数据 (类 Unix stat)

```
$ bstat bitfs://example.com/docs/readme.txt
File: readme.txt
Type: file
Hash: 3a7bd3e2...
Size: 4.2 KB
Owner: 02a1b2c3...
TxID: abc123...
Time: 2026-02-14 10:30:00 UTC
```

Access: free

```
$ bstat --versions bitfs://example.com/docs/report.pdf
v3 2026-02-14 10:30 a1b2c3... 1.5 MB (current)
v2 2026-02-13 15:00 d4e5f6... 1.2 MB
v1 2026-02-10 09:00 789abc... 1.0 MB
```

bcat – 输出文件内容到 stdout (类 Unix cat)

```
$ bcat bitfs://example.com/docs/readme.txt
# 自动解密 (免费数据用 Db=1 技巧, 付费数据用缓存的 key)

$ bcat --buy bitfs://example.com/premium/data.csv
# 自动购买 + 解密 + 输出 (purl 风格)
```

bget – 下载文件到本地 (类 wget/curl -O)

```
$ bget bitfs://example.com/docs/report.pdf
$ bget -o myreport.pdf bitfs://example.com/docs/report.pdf
$ bget --version 1 bitfs://example.com/docs/report.pdf # 下载特定版本
$ bget bitfs://alice@example.com/docs/report.pdf # Paymail 寻址

# 付费文件
$ bget bitfs://example.com/premium/data.csv
Price: 500 sat (50 sat/KB × 10 KB)
Use --buy to purchase and download.

$ bget --buy bitfs://example.com/premium/data.csv
Price: 500 sat (50 sat/KB × 10 KB) Purchasing...
Key saved. Downloading... Done.
# 后续 bget/bcat 自动使用缓存的密钥
```

btree – 递归目录树 (类 Unix tree)

```
$ btree bitfs://example.com/
example.com/
├── docs/
│   ├── readme.txt (4.2 KB) [free]
│   └── images/
│       └── logo.png (12 KB) [free]
├── premium/
│   └── data.csv (10 KB) [paid: 50 sat/KB]
└── LICENSE (1.1 KB) [free]
```

所有 b* 工具通用选项

```
--json          JSON 输出 (Agent 友好)
--no-cache      禁用本地缓存
--timeout N     请求超时 (秒)
--offline       强制只用缓存
```

九、bitfs 命令 (读写/Owner)

文件操作

```
bitfs put <local> <remote>  # 上传 (默认 D_node=1 加密 = 公开免费)
bitfs put --encrypt <l> <r>  # 上传并加密 (私有, encrypted=true)
bitfs put --keyword "tag1 tag2" --description "描述" <l> <r> # 附加元信息
bitfs mkdir <path>          # 创建目录
bitfs mv <src> <dst>         # 移动 (新节点 + 旧节点变 SOFT 链接, 重新加密)
bitfs cp <src> <dst>         # 复制 (独立新节点, 重新加密, 新 key_hash)
bitfs rm <path>              # 删除: (1) SelfUpdate 父目录移除 ChildEntry, (2) 花费目标节点 UTXO 到 fee 地址
bitfs rm -r <path>           # 递归删除目录 (先递归删除所有子节点)
bitfs rmdir <path>           # 删除空目录
bitfs link <target> <name>    # 硬链接 (复用 P_node, 仅添加 ChildEntry)
bitfs link -s <target> <name> # 软链接 (本 Vault, 创建 LINK 节点)
bitfs link -s example.com/path <name> # 远程软链接 (跨用户)
```

加密管理

```
bitfs encrypt <path>        # 公开 → 私有 (re-encrypt, encrypted=true, 新 key_hash)
bitfs decrypt <path>        # 私有 → 公开 (用 D_node=1 重新加密, encrypted=false, 新 key_hash)
```

交易 (Sell/Buy)

```
bitfs sell <path> --price <sat/KB>      # 标价出售 (更新 Metanet 元数据)
bitfs sell <path> --price <sat/KB> --recursive # 递归标价整个目录
bitfs sales [path]                      # 查看销售历史
```

sell 设置 price_per_kb 字段。总价由客户端计算: `ceil(price_per_kb × file_size / 1024)`。

发布

```
bitfs publish <domain> [path] # 绑定域名 (引导配置 DNS TXT + 更新 Metanet domain 字段)
bitfs unpublish <domain>      # 解除绑定
bitfs publish                  # 查看所有绑定关系
```

Daemon

```
bitfs daemon start [-d]    # 启动 (-d 后台)
bitfs daemon stop          # 停止
bitfs daemon status        # 状态
bitfs daemon config        # 显示当前配置
```

详细设计: 完整命令参考手册 → [九-B](#)

十、Shell 交互模式 (FTP 风格)

BitFS Shell 参照 FTP 客户端设计, 操作链上 Unix 文件系统:

```
$ bitfs shell
Wallet: 02a1b2... Balance: 0.05 BSV Vault: personal
BitFS Shell v0.1.0 Type 'help' for commands.

bitfs /docs>
```

Shell 命令 (FTP 风格)

```
=== Remote Navigation (链上) ===
ls [path]                列出目录内容
cd <path>                 切换远程目录
pwd                      显示远程当前路径
tree [path] [-d N]        树形显示
stat <path>               节点详细信息

=== Local Navigation (本地) ===
lcd <path>                切换本地目录
lpwd                     显示本地当前路径
lls [path]                列出本地文件

=== Transfer ===
```

```

get <remote> [local]          下载远程文件到本地
mget <pattern>                批量下载
put <local> [remote]          上传本地文件到远程
mput <pattern>                批量上传
put --encrypt <local> [remote] 加密上传

=== Remote File Operations ===
cat <file>                    输出远程文件内容（自动解密）
cp <src> <dst>                复制
mv <src> <dst>                移动/重命名
rm <path>                     删除
mkdir <path>                  创建目录
rmdir <path>                  删除空目录
link <target> <name>          硬链接
link -s <target> <name>       软链接

=== Encryption ===
encrypt <path>                公开 → 私有
decrypt <path>                私有 → 公开

=== Trading ===
sell <path> --price <sat/KB>  标价出售 [--recursive]
sales [path]                  查看销售历史

=== Wallet ===
balance                       查看余额
fund                           显示充值地址

=== Publishing ===
publish <domain> [path]       绑定域名
unpublish <domain>            解除绑定
publish                        查看绑定关系

=== Vault ===
vault list                    列出 Vault
vault use <name>              切换 Vault

=== Session ===
! <cmd>                       执行本地 shell 命令
help                           帮助
history                         命令历史
exit / quit / bye             退出

```

十一、买卖交易 – HTLC 原子交换

设计原则

- Seller 完全无状态: 不维护买家数据库, 不记录谁买过什么, 每次请求独立处理
- 重复购买 = 重复收费: Seller 不做去重
- Buyer 负责缓存: 购买后 key_capsule 在链上 (HTLC 揭示), 本地缓存到 ~/.bitfs/cache/keys/
- 后续访问不经过 Seller: Buyer 直接从 Seller daemon 拉加密数据 + 本地解密
- 一期不需要支付通道: BSV 链上 HTLC 手续费极低, 单次购买场景足够。流媒体/大文件微支付场景的支付通道见 [Metanet Chain 设计](#) (CDN 阶段引入)

Method 42 握手协议

Buyer 和 Seller 建立连接时, 使用 Method 42 ECDH 进行双向身份验证:

握手流程 (Buyer → Seller):

1. Buyer 发送: P_buyer (买家公钥) + nonce_b (随机数) + timestamp
2. Seller 发送: P_seller (卖家公钥, 即文件 Owner 的 P_node) + nonce_s + timestamp
3. 双方独立计算共享密钥:
Buyer: $\text{shared} = D_{\text{buyer}} \times P_{\text{seller}} \rightarrow \text{session_key} = \text{SHA256}(\text{shared.x} || \text{nonce_b} || \text{nonce_s})$
Seller: $\text{shared} = D_{\text{seller}} \times P_{\text{buyer}} \rightarrow \text{session_key} = \text{SHA256}(\text{shared.x} || \text{nonce_b} || \text{nonce_s})$
4. 双方用 session_key 发送 HMAC(session_key, "verify") 互相验证
5. 后续通信用 session_key 加密 (AES-256-GCM)

身份验证:

- Seller 的 P_seller 必须与 DNSLink 发布的 P_node 一致 (Buyer 可从 DNS TXT 验证)
- ECDH 天然保证: 只有私钥持有者才能计算出正确的 shared secret
- 中间人无法伪造: 没有 D_seller 就无法计算 session_key

HTLC 原子交换流程

1. Buyer 通过 bget --buy 发起购买
2. Buyer 与 Seller daemon 握手 (Method 42 ECDH, 见上)
3. Buyer 获取文件元数据 (price_per_kb, file_size)
4. Buyer 计算总价: $\text{total} = \text{ceil}(\text{price_per_kb} \times \text{file_size} / 1024)$
5. Buyer 请求 capsule_hash
 - Seller 计算: $\text{capsule} = \text{ECDH}(D_{\text{node}}, P_{\text{buyer}})$ 相关密钥材料
 - Seller 计算: $\text{capsule_hash} = \text{SHA256}(\text{capsule})$
 - Seller 返回: capsule_hash + payment_address

6. Buyer 创建 HTLC 交易:

IF (SHA256(preimage) == capsule_hash AND Sig(seller)) → seller 可领取
ELSE IF (timeout expired AND Sig(buyer)) → buyer 可退款

7. Buyer 广播 HTLC 交易 (htlc_tx 为必填字段)

8. Seller 验证 HTLC 交易已在链上 (mempool 或已确认) 后, 揭示 capsule (preimage) 领取付款
→ capsule 作为 preimage 出现在链上

9. Buyer 从链上获取 capsule → 派生解密密钥 → 解密文件
→ 缓存 key 到 ~/.bitfs/cache/keys/

后续访问:

Buyer 直接 bget → 从 Seller daemon 获取加密数据 → 本地解密 (已有 key, 不再付费)

原子性间隙: Buyer 广播 HTLC 后、Seller 返回 capsule 前存在竞态窗口。若 Seller 崩溃, Buyer 需等待 HTLC 超时 (默认 144 块, 约 24 小时) 后退款。缓解: Seller daemon 应监听 mempool, 确认 HTLC 交易存在后自动揭示 capsule, 无需依赖 Buyer 的显式通知。htlc_tx 字段为必填, Seller 必须验证链上交易后再返回 capsule。

Token 批量购买系统 (Hash Chain)

基于专利 US 2021/0399898 A1 [0189]-[0202] 的 Hash Chain 设计。适用于批量购买多个文件的访问权。

概念

Buyer 一次性预购 N 个 Token, 每个 Token 对应一个文件的解密密钥。Token 通过 Hash Chain 生成:

$$T_i = H^{(N-i)}(Y), i \in [1..N]$$

其中 Y 是 Buyer 的秘密种子

$$\text{链式关系: } H(T_1) = H^N(Y) = I_{\text{Bob}}, H(T_2) = T_1, \dots, H(T_N) = T_{N-1}$$

每次兑换揭示下一个 Token, 自动验证链条完整性。

Phase 1: Token 发行 (一次性)

Bob (买家) 准备:

1. 秘密种子 Y
2. N 个 token: $T_i = H^{(N-i)}(Y)$
3. 初始化值: $I_{\text{Bob}} = H^N(Y)$

Alice (卖家) 准备:

1. N 个秘密 (解密密钥): $X_1, X_2, \dots, X_N$
2. 初始化值: $I_{\text{Alice}} = \text{random } k$

原子交换:

Alice $\rightarrow$ Bob:

Output: [Hash Puzzle $H(I_{\text{Alice}})$] [Hash Puzzle $H(I_{\text{Bob}})$] [CheckSig P_{Bob}]
Value: x

Bob $\rightarrow$ Alice:

Output: [Hash Puzzle $H(I_{\text{Alice}})$] [Hash Puzzle $H(I_{\text{Bob}})$] [CheckSig P_{Alice}]
Value: $N \times \text{price} + x$

双方花费 $\rightarrow$ 揭示 I_{Alice} 和 $I_{\text{Bob}} \rightarrow$ Token 发行完成

Phase 2: Token 兑换 (逐个)

第 i 次兑换 (Bob 用 T_i 换取 X_i):

Bob $\rightarrow$ Alice:

Output: [Hash Puzzle $H(X_i)$] [Hash Puzzle $H(T_i)$] [CheckSig P_{Alice}]

Alice $\rightarrow$ Bob:

Output: [Hash Puzzle $H(X_i)$] [Hash Puzzle $H(T_i)$] [CheckSig P_{Bob}]

双方花费 $\rightarrow$ 揭示 X_i (解密密钥) 和 T_i (Token)

验证: $H(T_i) == T_{\{i-1\}}$ (或 I_{Bob} for $i=1$)

与 HTLC 的关系

- HTLC: 单次购买 (简单, 适合偶尔购买)
- Token: 批量预购 (高效, 适合订阅/批量场景)
- 两者共存, Token 是 HTLC 的泛化

目录树购买 (BIP32 xpub 解锁)

基于 BIP32 非硬化派生的 ECDH 传递性, 一笔 HTLC 解锁整棵目录树。

数学基础

BIP32 非硬化: $D_{\text{child}} = D_{\text{parent}} + \text{offset}$

$\text{offset} = \text{HMAC-SHA512}(\text{chaincode}, P_{\text{parent}} || \text{index})[:32]$

ECDH 传递性:

$S_{\text{child}} = D_{\text{child}} \times P_{\text{buyer}}$
 $= (D_{\text{parent}} + \text{offset}) \times P_{\text{buyer}}$

$$= D_{\text{parent}} \times P_{\text{buyer}} + \text{offset} \times P_{\text{buyer}}$$

$$= S_{\text{parent}} + \text{offset} \times P_{\text{buyer}}$$

→ Buyer 只需 $S_{\text{parent}} + x_{\text{pub_parent}}$ 即可推导所有非硬化子节点的 ECDH 密钥

购买流程

```

Buyer                                     Seller
|                                         |
|-- bget --buy /premium/ ----->|
|                                     |
|   Seller:                             |
|   1. 遍历 /premium/ 非硬化子节点      |
|   2. capsule = ECDH(D_dir, P_buyer)    |
|   3. 总价 = sum(非硬化子文件价格)      |
|                                     |
|<--- xpub_dir + capsule_hash -|
|                                     |
|-- HTLC (总价) ----->| 一笔交易
|                                     |
|<--- capsule (preimage) -----| 一个 capsule
|                                     |
Buyer 本地 (无需再请求 Seller):
for each non-hardened child:
    offset = HMAC(chaincode, P_dir || index)[:32]
    S_child = capsule + offset × P_buyer
    aes_key = KDF(S_child, key_hash_child)
    decrypt(content, aes_key) ✓

```

硬化 vs 非硬化 = 访问控制

BIP32 的硬化/非硬化派生成为第一等访问控制机制:

/premium/	← PAID 目录	
├─ chapter1.md	(非硬化, index 1)	← 目录购买包含 ✓
├─ chapter2.md	(非硬化, index 2)	← 目录购买包含 ✓
├─ chapter3.md	(非硬化, index 3)	← 目录购买包含 ✓
├─ bonus.md	(硬化, index 1')	← 需单独购买 ✗
└─ vip-content/	(硬化, index 2')	← 整个子目录排除 ✗

创建文件时选择:

```

bitfs put chapter4.md /premium/      # 默认非硬化 → 目录购买者可解密
bitfs put bonus2.md /premium/ --exclusive # 硬化 → 需单独购买

```

订阅模型

非硬化子节点的特性: 买了目录后, 未来新增的非硬化子节点也能解密 (Buyer 有 xpub, 能派生任意新 index)。

- 订阅 /magazine/ → 新期刊 (非硬化) 自动可读
- 特别期刊 (硬化) 需单独付费
- 配合 CLTV 实现限时订阅

xpub 传输

P_node 链上仍为 33 字节压缩公钥。xpub (含 chain code) 在购买协议中传输:

链上: P_node (33 bytes, 不变)
购买时: Seller 通过握手协议提供 xpub_dir (pubkey + chaincode)

与单文件购买的关系

方式	适用场景	HTLC 次数	Capsule
单文件 HTLC	偶尔购买一个文件	1 次/文件	单个
Token 批量	批量购买多个独立文件	1 次 setup	N 次兑换
目录树 xpub	购买整个目录/订阅	1 次	1 个 (xpub 派生)

十一-b、Rabin 签名

概述

Rabin 签名用于内容认证, 可在 Bitcoin Script 内直接验证。与 ECDSA 的互补关系: - ECDSA: 签整个交易 (Metanet Edge 创建, 已有) - Rabin: 签任意数据片段 (内容认证, 新增)

密钥

私钥: (p, q) , 两个大素数, $p \equiv 3 \pmod{4}$, $q \equiv 3 \pmod{4}$
公钥: $n = p \times q$

签名

1. 选择填充 U 使得 $H(m||U)$ 是模 n 的二次剩余
 2. $S = \text{sqrt}(H(m||U)) \bmod n$ (中国剩余定理计算)
- 签名 = (S, U)

验证 (Bitcoin Script)

$S^2 \bmod n == H(m||U)$?
只需平方 + 模运算, Script 内高效可实现

用途

1. 内容真实性: Owner 用 Rabin 签名内容, 链上可验证
2. 分片完整性: 链上分片内容每个 chunk 携带 Rabin 签名
3. 第三方认证: 审计方、认证机构可签名数据, 不需要是节点 Owner

Protobuf 字段

- `rabin_signature` (field 33): Rabin 签名 (S, U) 序列化
- `rabin_pubkey` (field 34): Rabin 公钥 n

十一-c、区块高度权限 (CLTV)

利用 `OP_CHECKLOCKTIMEVERIFY` 实现时间限制访问。

原理

在 Token 或数据交易的锁定脚本中加入 CLTV:

```
<block_height> OP_CHECKLOCKTIMEVERIFY OP_DROP  
<encrypted_content_or_key> OP_DROP  
OP_DUP OP_HASH160 <H160(P)> OP_EQUALVERIFY OP_CHECKSIG
```

效果: 此 UTXO 在 `block_height` 之前不可花费。

应用场景

1. 限时访问: Token/密钥的 UTXO 带 CLTV, 限制花费时间窗口

2. 定时发布: 内容在某区块高度后才可获取
3. 订阅模式: 按周期的 Token, CLTV 控制有效期
4. 版本控制: 旧版本数据在某高度后自动“过期” (不可再花费其 UTXO)

Protobuf 字段

`cltv_height` 字段 (field 35, uint32): 信息性标记, 实际约束在链上脚本中执行。

十一-d、内容分片与重组

适用场景

链上存储模式 (onchain=true) 中, 大文件需要分片到多笔数据交易。

分片规则

单交易内容上限: 由矿工策略决定 (BSV 当前无协议限制, 建议 $\leq 10\text{MB}/\text{tx}$)
分片方式: 按固定大小切分加密后的内容

交易结构

主 chunk (chunk_index=0): 由 Metanet 节点交易的 content_txids[0] 引用
后续 chunk: content_txids[1], content_txids[2], ...

每笔数据交易:

Input: Sig(D_node)

Output: <encrypted_chunk_N> OP_DROP [CheckSig P_node]

重组

客户端按 chunk_index 顺序拼接: $\text{decrypted} = \text{chunk}_0 \parallel \text{chunk}_1 \parallel \dots \parallel \text{chunk}_{\{N-1\}}$
验证: $\text{SHA256}(\text{SHA256}(\text{plaintext})) == \text{key_hash}$
验证: $\text{H}(\text{chunk}_0 \parallel \text{chunk}_1 \parallel \dots) == \text{recombination_hash}$

Protobuf 字段

- `chunk_index` (field 30): 本 chunk 序号 (0-based)
- `total_chunks` (field 31): 总 chunk 数 (0 = 非分片)

- `recombination_hash` (field 32): H(所有 chunk 拼接) 完整性校验
- `content_txids` (field 28): 所有数据交易 TxID 列表

十一-e、数据压缩

支持的压缩方案

```
COMPRESS_NONE = 0    (无压缩)
COMPRESS_LZW  = 1    (LZW, 适合文本)
COMPRESS_GZIP = 2    (GZIP, 通用)
COMPRESS_ZSTD = 3    (Zstandard, 高效)
```

流程

```
上传: plaintext → 压缩 → 加密 → 存储
下载: 获取 → 解密 → 解压 → plaintext
```

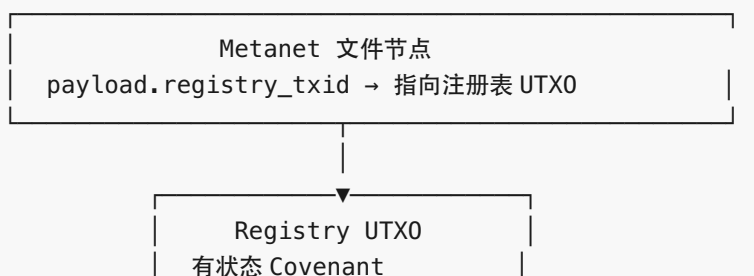
`compression` 字段 (field 29) 标记使用的压缩方案, 客户端据此选择解压算法。

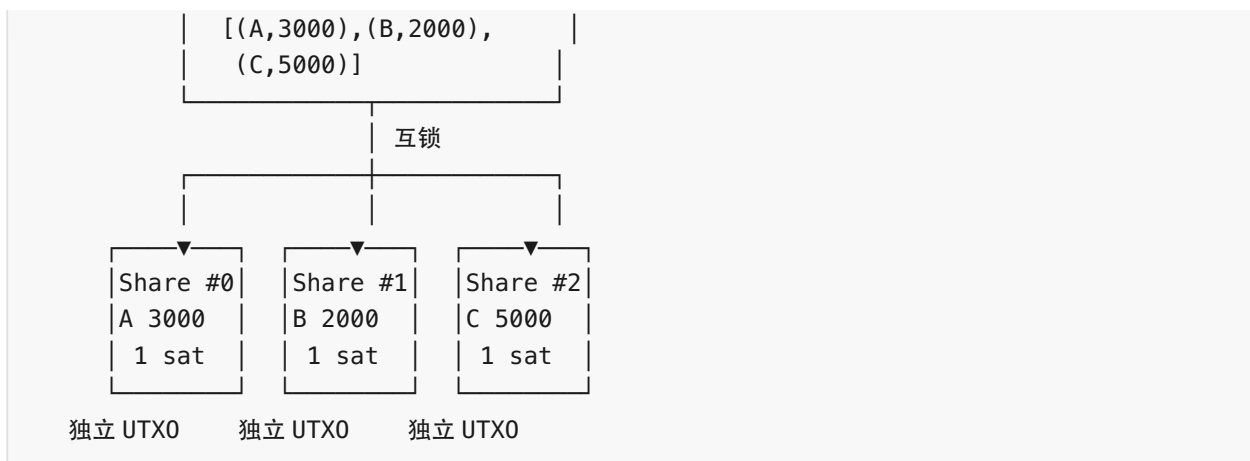
十一-f、收益权表与 ISO (Initial Share Offering)

概述

文件创作者可选择将文件收益权证券化: 通过 ISO (Initial Share Offering) 公开发售收益权份额。份额以 UTXO 形式存在, 可自由转让、拆分、合并。购买文件时, 收益通过 Covenant 脚本自动按份额分配给所有股东。

双层架构





- Share UTXO (份额证书): 每个份额是独立的链上 UTXO, 持有即拥有。可自由转让、拆分、合并。
- Registry UTXO (注册表): 有状态 Covenant, 记录当前所有股东地址和份额。分账时读取此表。
- 互锁: 两种 Covenant 互相验证对方存在于同一笔交易中, 确保注册表与份额 UTXO 永远一致。

份额总量

创作者自定义总量 (类似公司决定发行股数):

音乐单曲: 发行 10,000 shares
 电影项目: 发行 1,000,000 shares
 小文章: 发行 100 shares

Covenant 守恒规则: `sum(output_shares) == sum(input_shares)`, 不关心总量具体数值。

Covenant 脚本

Share UTXO Covenant (份额证书):

```

ShareCovenant(node_id, share_amount):
  <holder_sig> <holder_pubkey> →
  1. 验证持有者身份 (P2PKH)
  2. OP_PUSH_TX: 验证 Input[0] 是 Registry UTXO (互锁)
  3. 验证输出份额总和 == 输入份额 (守恒, 允许拆分/合并)
  4. 验证输出仍绑定同一 node_id
  
```

Registry UTXO Covenant (注册表):


```
RegistryCovenant(node_id, entries[]):
  MODE_TRANSFER:
    1. 验证有 Share UTX0 作为 input (互锁)
    2. 验证新注册表状态与份额变更一致
    3. 总份额守恒
    4. 输出新 Registry UTX0 (状态更新)

  MODE_DISTRIBUTE:
    1. 验证有 HTLC 支付作为 input
    2. 按 entries[] 验证每个输出金额
    3. 输出新 Registry UTX0 (状态不变, 原样传递)
```

四种核心交易

TX1: ISO Genesis (份额发行)

```
ISO Genesis Tx:
  Input 0: 创作者 Fee UTX0

  Output 0: Registry UTX0 [(A,4000), (ISO_POOL,6000)] → 1 sat
    元数据: total=10000, price=100sat, status=OPEN
  Output 1: Share UTX0 #0 (node_id, 4000, addr_A) → 1 sat (创作者自留)
  Output 2: ISO Pool UTX0 (6000 shares, 100 sat/份) → 1 sat (公开发售池)
  Output 3: Change
```

ISO Pool 是特殊 Covenant — 任何人都可花费, 只要满足条件:

```
ISOPoolCovenant(node_id, remaining, price, creator_addr):
  1. 验证支付: buy_amount × price → creator_addr
  2. 验证份额: buy_amount ≤ remaining
  3. 输出:
    - Registry UTX0 (更新: 加入 buyer)
    - Share UTX0 (buyer, buy_amount)
    - ISO Pool UTX0 (remaining - buy_amount) (如有剩余)
    - P2PKH → creator_addr (payment)
```

TX2: ISO Buy (购买份额, 原子交换)

```
ISO Buy Tx:
  Input 0: Registry UTX0 (旧: [(A,4000), (ISO_POOL,6000)])
  Input 1: ISO Pool UTX0 (6000 available)
  Input 2: Buyer 支付 UTX0 ← Buyer 签名

  Output 0: Registry UTX0 (新: [(A,4000), (Buyer,500), (ISO_POOL,5500)])
  Output 1: Share UTX0 (Buyer, 500) → 1 sat
```

```
Output 2: ISO Pool UTX0 (5500 remaining) → 1 sat
Output 3: P2PKH → creator_addr → 50000 sat (500 × 100)
```

TX3: 份额转让 / 拆分

Transfer Tx:

```
Input 0: Registry UTX0 (旧状态)
Input 1: Share UTX0 (A, 3000) ← A 签名

Output 0: Registry UTX0 (新状态: A→D)
Output 1: Share UTX0 (D, 3000) → 1 sat
```

Split Tx (拆分):

```
Input 0: Registry UTX0
Input 1: Share UTX0 (A, 3000) ← A 签名

Output 0: Registry UTX0 (更新)
Output 1: Share UTX0 (A, 2000) → 1 sat (A 保留)
Output 2: Share UTX0 (D, 1000) → 1 sat (D 获得)
```

TX4: 购买文件, 收益自动分配

Purchase Tx:

```
Input 0: Buyer HTLC payment
Input 1: Registry UTX0 (读取状态) ← Seller 签名

Output 0: Registry UTX0 (不变, 传递) → 1 sat
Output 1: P2PKH → addr_A (payment × 2000 / 10000) → 收益
Output 2: P2PKH → addr_D (payment × 1000 / 10000) → 收益
Output 3: P2PKH → addr_B (payment × 2000 / 10000) → 收益
Output 4: P2PKH → addr_C (payment × 5000 / 10000) → 收益
```

二级市场: 份额原子交换

Atomic Swap Tx:

```
Input 0: Registry UTX0
Input 1: Share UTX0 (A, 1000) ← A 签名
Input 2: D 的支付 UTX0 ← D 签名

Output 0: Registry UTX0 (更新: A→D)
Output 1: Share UTX0 (D, 1000) → 1 sat (份额给 D)
Output 2: P2PKH → addr_A → 5000 sat (钱给 A)
```

一笔交易, 要么全部成功, 要么全部失败。无需信任中介。

ISO 关闭

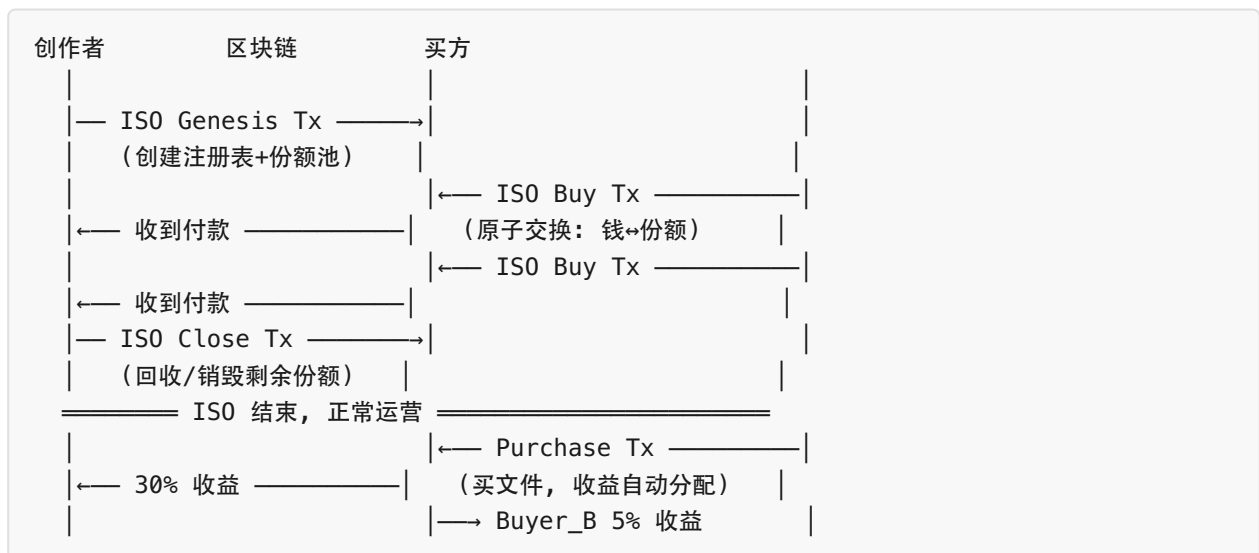
创作者可随时关闭 ISO (回收未售出份额):

```
ISO Close Tx:
Input 0: Registry UTX0 ([A,4000), (Buyer,500), (ISO_POOL,5500)])
Input 1: ISO Pool UTX0 (5500 remaining) ← creator 签名

Output 0: Registry UTX0 ([A,9500), (Buyer,500)])
Output 1: Share UTX0 (A, 9500)
```

也可选择销毁未售出份额 (缩减总量), 让已售份额更值钱。

ISO 生命周期



与现有系统的关系

组件	关系
HTLC	Purchase Tx 中的支付部分复用 HTLC
Token	Token 购买也触发 Registry 分账
Metanet	registry_txid 存储在 Metanet 节点 Protobuf 中
Method 42	解密密钥分发不变, 收益分配是独立层
Rabin	可用于签名 Registry 状态变更

CLI 命令

```
# ISO 发行
bitfs iso create <path> --total-shares 10000 --reserve 4000 --price 100
bitfs iso close <path>      # 关闭 ISO, 回收/销毁剩余份额
bitfs iso status <path>     # 查看 ISO 状态

# 份额管理
bitfs share list <path>     # 查看收益权表
bitfs share transfer <path> --to <addr> --amount 1000
bitfs share split <path> --amounts 2000,1000

# 二级市场
bitfs share sell <path> --amount 500 --price 5000 # 挂卖
bitfs share buy <path> --from <addr> --amount 500 # 原子交换购买
```

十二、版本控制

Metanet 内置版本控制

同一 P_node + 不同 TxID = 不同版本。 排序规则: 1. 不同区块: 区块高度更高 = 最新版本 2. 同一区块: TTOR 靠后 = 最新版本

```
$ bstat --versions bitfs://example.com/docs/report.pdf
$ bget --version 1 bitfs://example.com/docs/report.pdf
```

Git Remote Helper

`git-remote-bitfs` 自定义 remote helper, 实现 `import / export capabilities`, 让 BitFS 成为完整的 git remote:

```
git remote add origin bitfs://example.com/projects/myapp
git push origin main      # fast-export → packfile → 加密 → 上传
git clone bitfs://example.com/projects/myapp # 下载 → 解密 → fast-import
git fetch origin
git pull origin main
```

URL 方案

```
bitfs://domain/path/to/repo
```

- domain — DNS 域名 (通过 `_bitfs_pubkey` TXT + `_bitfs._tcp` SRV 解析到 daemon)
- path — BitFS 文件系统中的目录路径, 该目录作为 git 仓库的根

仓库目录结构

一个 git 仓库在 BitFS 中表现为普通目录, 内含特殊子节点:

<code>/projects/myapp/</code>	← 仓库根目录 (普通 DIR 节点)
<code>.git-packs/</code>	← DIR: 存放 packfile
<code>pack-<hash1>.pack</code>	← FILE: 加密的 git packfile
<code>pack-<hash2>.pack</code>	← FILE: 增量 push 产生的新 packfile
<code>...</code>	
<code>.git-refs</code>	← FILE: refs 映射表 (JSON)
(其他 BitFS 文件)	← 仓库目录也可包含非 git 文件

`.git-refs` 内容:

```
{
  "HEAD": "ref: refs/heads/main",
  "refs/heads/main": "abc123...",
  "refs/heads/dev": "def456...",
  "refs/tags/v1.0": "789abc..."
}
```

import/export 协议

helper 声明的 capabilities:

```
capabilities
import
export
refspec refs/heads/*:refs/bitfs/heads/*
refspec refs/tags/*:refs/bitfs/tags/*
```

Push 流程 (export): 1. git 发送 fast-export 流 (commits, trees, blobs) 到 helper stdin 2. helper 写入临时 bare repo → 打包为 packfile 3. Method 42 加密 packfile 4. 上传到 BitFS `.git-packs/pack-<hash>.pack` 5. 更新 `.git-refs` (新 commit SHA → ref 映射) 6. stdout 回复 `ok refs/heads/main`

Clone/Fetch 流程 (import): 1. helper 读取远程 `.git-refs` → 返回 refs 列表 2. 下载所有 packfiles (可能触发 buy/HTLC 付费) 3. Method 42 解密 4. 生成 fast-import 流 → 写入 stdout 5. `git fast-import` 重建本地对象

增量 Push: 每次 push 不覆盖已有 packfile, 而是新增。push 前读取远程 `.git-refs` 获取已有 commit SHA, 通过 `git rev-list --objects <local> ^<remote>` 计算新 objects, 只打包这些。

增量 Fetch: 对比本地缓存的 packfile 哈希列表 (`.git/bitfs/`), 只下载新 packfile。

加密与付费

所有 packfile 默认 Method 42 加密, 三种访问级别:

级别	场景	行为
OWNER	repo owner fetch/clone	直接解密, 无需付费
DESIGNATED	ACL 授权的协作者	群加密解密, 无需付费
PAID (sell)	任意人 clone	buy 流程: HTLC 付费 → capsule → 解密

clone 付费流程: 元数据 (refs) 免费读取 → 下载 packfile 触发 buy → Method 42 握手 → 计算总价 → 用户确认 → HTLC 原子交换 → 解密。

多人协作

repo 目录的 ACL 控制写权限:

```
# Owner 授权
bitfs acl /projects/myapp --add-write bob_pubkey

# Bob push (被授权)
git push origin main # 群签名证明 write 组成员身份, daemon 验证后接受

# Eve push (未授权)
git push origin main # error: permission denied
```

冲突处理与标准 git 一致: helper push 前检查远程 ref 是否是本地的祖先, 非 fast-forward 时拒绝。

原子性保证

先上传 packfile, 后更新 refs。如果 refs 更新失败, packfile 成为孤立数据 (无害, 可 gc 清理)。

Daemon 新增端点

```
POST /_bitfs/git/push          ← 接收 packfile + refs 更新
  Request: multipart (packfile binary + refs JSON)
  Auth:    Method 42 ECDH 握手
  权限:    ACL write 组验证 (群签名)
  Response: { "ok": true, "pack_hash": "..."}

GET /_bitfs/git/refs/{path}   ← 读取仓库 refs (免费)
  Response: { "HEAD": "ref: refs/heads/main", ... }
```

通信方式

```
Owner (本地有 ~/.bitfs/):
  git-remote-bitfs → 直接读写 ~/.bitfs/data/ + 本地钱包签名 Metanet 交易

Remote (通过 daemon):
  git-remote-bitfs → HTTP → daemon
    GET /_bitfs/git/refs/...    (读 refs)
    GET /_bitfs/data/...        (下载 packfile, 可能 x402)
    POST /_bitfs/git/push       (上传 packfile + 更新 refs)
```

配置

```
git config bitfs.home ~/.bitfs      # BitFS 主目录 (默认 ~/.bitfs)
git config bitfs.autoConfirm true    # 自动确认付费 (agent-first)
git config bitfs.vault default       # 使用的 vault
```

BitFS 天然就是 Git LFS — Metanet 存元数据指针, 实际内容在内容寻址存储, 不论文件大小。不需要单独的 LFS 机制。

十三、Daemon 配置 (LFCP)

Daemon 角色 — Local Full-Copy Peer (LFCP)

Daemon 作为 LFCP (Local Full-Copy Peer), 是 Owner 节点数据的本地完整副本服务。任何第三方也可以为公开内容运行 LFCP。

1. 内容存储与服务 — 缓存 Owner 节点的 Metanet 交易和内容, 对外提供 HTTP 服务

- 链下内容 (默认): 从本地存储服务加密内容
- 链上内容: 从区块链交易中提取加密内容 (解析 OP_DROP 数据交易)

2. Metanet 元数据服务 – 为 Visitor 提供 Metanet 树结构查询 (SPV 模式下 Visitor 不查链)
3. x402 网关 – 收取 CDN 带宽费
4. HTLC/Token 处理 – 处理 sell/buy: 握手 → 提供 capsule_hash → 揭示 capsule 领款; Token 批量购买
5. WebMCP + Agent 支持 – 为浏览器 Agent 和 CLI Agent 提供自描述接口
6. 公开内容镜像 – 任何第三方可运行 LFCP 缓存公开内容, 通过 SRV DNS 记录加入 CDN 负载均衡

HTTP API

监听地址: 标准 HTTP/HTTPS 端口 (80/443, 生产环境建议反向代理 + TLS)

GET /	根路径 (Content Negotiation: HTML/Markdown, 见 Agent 支持)
GET /{path}	路径访问 (Content Negotiation, 目录返回 index.html)
GET /_bitfs/data/{hash}	获取加密数据 (可能触发 x402)
GET /_bitfs/meta/{pnode}/{path}	查询 Metanet 元数据
GET /_bitfs/health	健康检查
POST /_bitfs/handshake	Method 42 ECDH 握手 (双向身份验证, 建立 session)
POST /_bitfs/pay/{invoice_id}	提交 BSV 交易 (x402 CDN 带宽费, 见下方验证流程)
GET /_bitfs/buy/{txid}	获取购买信息 (capsule_hash, 价格)
POST /_bitfs/buy/{txid}	提交 HTLC, Seller 揭示 capsule

POST /_bitfs/pay/{invoice_id} 验证流程

请求体:

```
{
  "raw_tx": "<hex>",      // BSV 原始交易 (必填)
  "merkle_proof": "<hex>"  // Merkle 证明 (已确认交易可选提供)
}
```

验证流程: 1. 解析 raw_tx, 提取输出列表 2. 验证存在匹配 invoice 金额和地址的输出 3. 验证交易已在 mempool 中或提供有效 Merkle proof 4. 验证通过后标记 invoice 为 PAID, 返回 200 + 内容

WebMCP + Agent 支持

BitFS daemon 通过两种方式支持 AI Agent 自动发现和使用:

1. WebMCP (浏览器 Agent)

Daemon 的 HTML 页面通过 WebMCP 声明式 API 注册 BitFS 工具:

```
<!-- 列目录 -->
<form toolname="bitfs_ls" tooldescription="List files and directories in a BitFS path">
  <input name="path" type="text" required placeholder="/docs/" />
</form>

<!-- 读取文件 -->
<form toolname="bitfs_cat" tooldescription="Read file content from BitFS (auto-decrypt)">
  <input name="path" type="text" required />
</form>

<!-- 获取文件元数据 -->
<form toolname="bitfs_stat" tooldescription="Get file/directory metadata including size, hash, access
type">
  <input name="path" type="text" required />
</form>

<!-- 购买付费文件 -->
<form toolname="bitfs_buy" tooldescription="Purchase and download a paid file">
  <input name="path" type="text" required />
  <input name="buyer_pubkey" type="text" required pattern="^[0-9a-f]{64}$" />
</form>
```

浏览器中的 AI Agent 通过 `navigator.modelContext` 自动发现这些工具，可直接调用。

2. Content Negotiation (CLI Agent)

根据 `Accept` 头返回不同格式:

```
Accept: text/html      → 正常 HTML 页面 (含 WebMCP 声明)
Accept: text/markdown  → Markdown 格式的 Agent 使用指南
Accept: application/json → JSON 格式元数据
```

当 Agent 请求 `text/markdown` 时，返回自描述文档:

```
# example.com -- BitFS Site

This site is powered by BitFS, a decentralized file system on BSV blockchain.

## Available Tools

Install BitFS CLI: `go install github.com/bitfs/cli/cmd/...@latest`

### Browse files
bls bitfs://example.com/      # list root directory
```

```

bls bitfs://example.com/docs/  # list subdirectory

### Read files
bcat bitfs://example.com/docs/readme.txt  # output to stdout
bget bitfs://example.com/docs/report.pdf  # download to local

### File info
bstat bitfs://example.com/docs/readme.txt  # metadata
btree bitfs://example.com/  # directory tree

### Purchase paid content
bget --buy bitfs://example.com/premium/data.csv

## Directory Contents
- docs/ (dir, 3 files)
- premium/ (dir, 1 file, paid: 50 sat/KB)
- LICENSE (1.1 KB, free)

```

这使 AI Agent (如 Claude, GPT) 能自动发现 BitFS 站点并学会如何访问。

x402 流程 (人类 vs Agent)

免费内容: 直接提供, 人类和 Agent 无差异。

付费内容: 402 响应根据请求者不同, 返回不同格式:

```

Client → GET /{path}

IF 免费 OR 已付费:
    200 OK + content (自动解密)

IF 需付费:
    402 Payment Required
    Headers:
        X-Price: 500                (总价 satoshis)
        X-Price-Per-KB: 50         (单价)
        X-File-Size: 10240         (文件大小 bytes)
        X-Invoice-Id: abc123       (发票 ID)
        X-Expiry: 1708000000       (过期时间)
    Body (Content Negotiation):
        Accept: text/html          → 付费墙 HTML 页面 (人类看到 "需付费" 提示)
                                   + WebMCP 声明 (浏览器 Agent 可发现 bitfs_buy 工具)
        Accept: text/markdown      → Markdown 说明 + CLI 命令:
                                   "This content requires payment: 500 sat
                                   Run: bget --buy bitfs://example.com/premium/data.csv"
        Accept: application/json    → JSON 结构化付费信息

Client (Agent) → POST /_bitfs/pay/{invoice_id} { "raw_tx": "<hex>", "merkle_proof":

```

```
"<hex>" }
```

Daemon 验证 raw_tx 输出匹配 invoice → 确认 mempool/Merkle proof → 200 OK + content

核心体验差异: - 人类 看到付费墙 HTML 页面 → 被拦截 (需要手动操作付费) - 浏览器 Agent 看到 WebMCP bitfs_buy 工具 → 自动调用付费 - CLI Agent 看到 bget --buy 命令 → 自动执行付费 - Agent 付费是无感的: Agent 有钱包, 直接完成 HTLC 交换, 人类甚至不知道发生了什么

配置文件 (~/.bitfs/config.toml)

统一配置文件, 包含全局设置和 daemon 配置 (以下示例为等效 YAML 格式便于阅读):

```
# 全局设置
network: mainnet
output: plain
cache:
  enabled: true
  max_size: "5GB"
  meta_ttl: 3600
  data_ttl: 86400
  eviction: lru

# Daemon 设置
daemon:
  listen: "0.0.0.0:80"
  tls: { enabled: false, cert: "", key: "" }
  x402:
    enabled: true
    price_per_mb: 100    # satoshis (CDN 带宽费)
    free_quota_mb: 10    # 每 IP 每天
    invoice_expiry: 300  # 秒
  security:
    rate_limit: { rpm: 60, burst: 20 }
    cors: { origins: ["*"], methods: ["GET", "POST"] }
    max_request_size: "10MB"
  storage:
    data_dir: "~/.bitfs/data" # 加密文件存储目录
    db_path: "~/.bitfs/daemon.db"
    cache_size: "1GB"
  log: { level: "info", file: "~/.bitfs/daemon.log" }
```

详细设计: HTTP API、握手协议、x402、HTLC → [十三-B](#)

十四、技术栈

组件	选择	包
语言	Go 1.21+	-
区块链	BSV	github.com/bitcoin/bitcoin (BSV Association 官方 Go SDK)
内容寻址存储	内建 (SHA-256 为键)	无外部依赖, 自实现
HTTP 服务	标准库 net/http	-
CLI	Cobra + Viper	github.com/spf13/cobra , github.com/spf13/viper
本地存储	BadgerDB	github.com/dgraph-io/badger /v3
测试	testify	github.com/stretchr/testify

十五、Go 项目结构

```
bitfs/
├── cmd/
│   ├── bls/main.go          # 独立只读工具
│   ├── bcat/main.go
│   ├── bget/main.go
│   ├── bstat/main.go
│   ├── btree/main.go
│   └── bitfs/main.go        # 主命令 (子命令 + shell)
```

```

├── internal/
│   ├── metanet/           # Metanet 解析、查询、创建节点
│   ├── method42/         # Method 42 加密引擎
│   │   ├── hdwallet.go   # HD 钱包 (BIP39/44 + passphrase)
│   │   ├── encrypt.go    # 文件加密/解密
│   │   ├── keycapsule.go # Key Capsule + HTLC
│   │   └── handshake.go  # Method 42 ECDH 握手协议
│   ├── storage/          # 内容寻址存储 (SHA-256 为键, 本地文件系统)
│   ├── spv/              # SPV 轻节点 (tx 存储 + Merkle proof + 区块头)
│   ├── dnslink/          # DNSLink 解析 + 双向验证
│   ├── x402/             # x402 支付协议
│   ├── daemon/           # Daemon 服务
│   ├── addressing/       # bitfs:// URI 解析
│   └── shell/            # FTP 风格交互 Shell
├── go.mod
└── go.sum

```

十六、错误处理

Exit Codes

```

0 = 成功      1 = 一般错误      2 = 参数错误
3 = 网络错误  4 = 数据验证错误  5 = 认证错误
6 = 未找到    7 = 支付错误

```

策略

- DNS/网络超时: 重试 3 次, 指数退避 (1s/2s/4s), 有缓存则提示 `-cached`
- UTXO 已花费: 自动重新构造交易 + 重试 (最多 3 次)
- 余额不足: 提示金额差额 + `bitfs wallet fund` 地址
- 内容哈希不匹配: 拒绝保存, 建议从其他来源重试
- 解密失败: 提示密钥错误或数据损坏
- 交易广播被拒: 显示拒绝原因
- HTLC 超时: 自动退款

交易费策略

BSV 费率是动态的。客户端需要: - 从 `go-sdk` 获取当前网络费率 (sat/byte) - 构造交易时按当前费率计算手续费, 从费用密钥链 (m/44' /236' /0') 支付 - 费率不足导致广播失败时, 用更高费率重构交易并重试

Daemon HTTP 错误码

400 请求格式错误	402 需要付费 (x402)
404 内容未找到	408 超时 409 交易冲突
429 限流	500 内部错误 503 存储不可用

JSON 错误格式 (-json)

```
{"error": { "code": "NETWORK_TIMEOUT", "message": "...", "retry": true, "cached": false } }
```

十七、离线模式

能力矩阵

操作	离线	条件
bls / bstat / btree / bcat	部分	有缓存可用
bget	否	需要网络连接 Seller daemon
bitfs put / rm / mv	否	需要 BSV 网络

选项

```
--offline    # 强制只用缓存
--cached     # 显示 [cached] 标记
--no-cache   # 禁用缓存
```

配置: `cache: { enabled: true, max_size: "5GB", meta_ttl: 3600, data_ttl: 86400, eviction: "lru" }`

十八、多设备

UTXO 冲突: 乐观并发 + 自动重试

设备 A 花费 UTXO_1 → 成功
设备 B 花费 UTXO_1 → 被拒 (double-spend)
→ 用新 UTXO 重新构造交易, 重试 (最多 3 次)

版本冲突: Last-Write-Wins

同一 P_node 多个版本, 按区块高度/TOR 确定最新。两个版本都保留, `bstat --versions` 查看。

Metanet Chain (去中心化 CDN) 设计已移至独立文档: [../metanet/](https://metanet/)

二十、远期功能

20.1 共享与权限 (远期: 群签名阶段)

share/unshare/chown 全部推迟到群签名 (Group Signature) 技术成熟后实现。

- share = 文件的所有权和修改权 (≠ sell/buy 的查看权)
- 群签名实现: Owner 创建群, 成员独立签名/解密, 无需多方协作
- share_list 字段 (payload field 22) 指向独立的 Metanet 权限节点

当前阶段只有 sell/buy (查看权交易)。

20.2 sCrypt 链上验证 (远期)

用 sCrypt 在 Bitcoin Script 中实现 EC 运算, 用于验证 capsule 和签名的链上有效性, 使交易无需 Seller 在线即可自动完成。注意: sCrypt 用于验证而非生成 — 密钥和签名仍在链下计算。当前用 HTLC/Token 方案 (需 Seller daemon 在线)。

二十一、会话管理 (Lock / Unlock)

问题

CLI 工具 (bls, bcat, bget, bstat, btree, bitfs 子命令) 是短生命周期进程，每次执行完即退出。当操作涉及私钥 (写入、读取 PRIVATE 内容、加密/解密) 时，需要用户输入密码来解锁 Wallet。频繁输入密码严重影响使用体验。

设计目标

- `bitfs unlock` 一 输入一次密码，后续 CLI 命令自动获取密钥，无需重复输密码
- `bitfs lock` 一 安全清除所有缓存的密钥材料
- 支持可选的超时自动锁定
- 与 `daemon` 是否运行无关，均可独立工作

命令接口

```
bitfs unlock          # 永不过期，直到 bitfs lock
bitfs unlock --duration 30m  # 30 分钟后自动过期
bitfs unlock --duration 8h   # 8 小时后自动过期

bitfs lock            # 立即锁定，安全清除所有密钥材料
```

混合模式

根据 `daemon` 是否运行，自动选择最优的密钥缓存策略：

`bitfs unlock` 流程：

```
bitfs unlock [--duration <duration>]
→ 提示输入密码
→ derived_key = Argon2id(password, salt_from_wallet_enc, ...)
→ 验证 derived_key 正确 (试解密 seed)
├─ daemon 运行中?
│  └─ 是 → 发送 unlock RPC (传递 derived_key) → daemon 在进程内存中持有 derived_key
│      后续 CLI 通过 Unix socket 请求签名/解密
│      derived_key 不落盘
└─ 无论 daemon 是否运行:
    └─ 写 session 文件 ~/.bitfs/session (权限 0600)
        仅记录 session_token + vault_id + 过期时间
        derived_key 不写入 session 文件
```


注意： session 文件始终写入， 但仅包含会话元数据， 不包含密码或 derived_key。当 daemon 在线时， CLI 优先走 daemon Unix socket 获取签名/解密服务；当 daemon 不在线且 session 文件有效时， CLI 需重新提示输入密码（单次使用）。

bitfs lock 流程：

```
bitfs lock
├─ daemon 运行中？
│   └─ 是 → 发送 lock RPC → daemon 清零内存中的 derived_key (memset zero)
└─ 无论 daemon 是否运行：
    └─ session 文件存在？
        └─ 是 → 安全覆写（3 次零填充 + fsync）→ 删除
```

两个动作独立执行、互不影响。lock 确保 derived_key 从 daemon 内存中清除， session 文件从磁盘上安全删除。

CLI 工具获取密钥的优先级：

```
CLI 启动 → 需要私钥操作
1. daemon 运行中且已 unlock？ → Unix socket 请求签名/解密服务 → 完成
2. session 文件存在且未过期？ → 确认会话有效，提示输入密码（derived_key 仅本次使用，用完即清零）→ 完成
3. 都不可用 → 提示输入密码（单次使用，derived_key 用完即清零）
```

Session 文件

路径： `~/.bitfs/session` 权限： `0600` （仅 owner 可读写）

格式：

```
{
  "session_token": "<随机生成的 session token (hex, 32 bytes)>",
  "vault_id": "personal",
  "created_at": 1739600000,
  "expires_at": 1739601800
}
```

字段	说明
<code>session_token</code>	随机生成的会话令牌，用于向 daemon 证明已授权（非密钥材料）

字段	说明
<code>vault_id</code>	当前活跃 vault 标识
<code>created_at</code>	Unix 时间戳, unlock 时写入
<code>expires_at</code>	Unix 时间戳。0 表示永不过期 (未指定 <code>--duration</code> 时)

安全原则: session 文件绝不存储密码、derived_key 或任何密钥材料。derived_key 仅存在于 daemon 进程内存中 (daemon 在线时), 或由 CLI 临时从用户输入派生 (daemon 离线时, 用完即清零)。

过期检查逻辑 (每个 CLI 工具启动时执行) :

```

读取 ~/.bitfs/session
→ 不存在 → session 无效
→ expires_at > 0 且 当前时间 > expires_at
  → 过期 → 安全覆写 (3 次零填充) + 删除 → session 无效
→ expires_at == 0 或 当前时间 ≤ expires_at
  → session 有效 (可用 session_token 与 daemon 通信, 或确认用户已授权)

```

安全覆写

删除 session 文件时执行 3 次零填充 (防止磁盘残留) :

```

for i := 0; i < 3; i++ {
  写满 0x00 到文件 (覆盖全部内容)
  fsync (强制刷盘)
}
os.Remove(文件)

```

限制: 在 SSD 上, 由于 wear leveling 机制, 覆写零填充不能保证物理擦除原始数据。这是“尽力而为”的安全措施。对高安全需求场景, 建议配合全盘加密 (FileVault / LUKS)。

安全对比

场景	derived_key 位置	安全级别	说明
daemon 运行 + unlock	daemon 进程内存	高	

场景	derived_key 位置	安全级别	说明
			derived_key 不落盘, 进程退出即消失
daemon 未运行 + unlock	无持久化 (session 文件仅含 token)	中高	CLI 每次需输密码临时派生, session 文件确认授权状态
未 unlock	无缓存	最高	每次操作单独输密码

与同类工具对比

工具	方式	密钥存储
ssh-agent	后台进程 + Unix socket	仅内存
gpg-agent	后台进程 + Unix socket	仅内存
Bitcoin Core	walletpassphrase RPC	长驻进程内存
sudo	时间戳文件	不存密钥, 只记录认证时间
HashiCorp Vault	token 文件 ~/.vault-token	session token
BitFS	混合: daemon 内存 + session token	daemon 内存 (优先, 存 derived_key) / session 文件 (仅 token, 不存密钥)

二十二、权限管理 (ACL + 群签名 / 群加密)

设计目标

为 BitFS 文件系统提供多用户协作的权限管理能力, 支持:

- POSIX ACL 风格的用户/分组权限控制 (r/w)
- 群签名 (Group Signature) 实现匿名写权限
- 群加密 (Group Encryption) 实现群体读权限
- 与 Method 42 无缝集成

权限模型

参照 Unix POSIX ACL，两级权限，不设 x 权限：

权限	含义	强制方式	技术
r (read)	可解密元数据和内容	密码学强制	群加密：用 GPK 加密，群成员可解密
w (write)	可更新元数据和内容	应用层强制	群签名：群成员独立签名，应用层验证

为何不需要 x 权限：在加密文件系统中，目录的 ChildEntry 列表本身是加密的。没有 r 权限（无法解密）就看不到子节点的 pubkey，自然无法遍历。r 已经是遍历的前提条件。

核心技术：群签名 + 群加密

群签名 (Group Signature)： - 群管理者 (Group Manager = Owner) 创建群，生成群公钥 GPK 和群管理密钥 GMK - Owner 为成员签发 credential（绑定到特定子群） - 任意成员可用自己的 credential 独立签名 $\rightarrow \sigma$ 对 GPK 验证通过 - 验证者无法知道具体签名者（匿名性） - Owner 可打开签名识别签名者（可追溯性） - 支持子群：credential 中编码子群属性，签名可证明所属子群

群加密 (Group Encryption)： - 群签名的对偶技术 - 用 GPK 加密，任意群成员可独立解密 - 支持子群级别加密：只有特定子群的成员能解密

统一框架：同一个群结构（GPK + credential）同时提供签名和加密能力。

三层加密体系

层级	技术	用途
credential 分发	Method 42 (ECDH)	Owner $\rightarrow$ 成员，一对一加密
内容加密/解密	群加密 (GPK)	一对多，任意成员可解密
写入授权	群签名 (GPK)	任意成员可签名，应用层验证

ACL 节点

ACL 存储在独立的 Metanet 节点中，P_node = PK_owner（仅 Owner 可修改）。

ACL Node（独立 Metanet 节点）：

P_node: PK_owner

← 只有 owner 能修改

```

owner:      PK_alice          ← 所有者身份
group_pk:   GPK              ← 群公钥 (签名 + 加密共用)

access_acl:                                ← 当前节点的访问权限
- { tag: USER_OBJ, perms: rw }          ← owner 权限
- { tag: USER, pubkey: PK_bob, perms: rw }
- { tag: GROUP, subgroup: "editors", perms: rw }
- { tag: GROUP, subgroup: "viewers", perms: r }
- { tag: MASK, perms: rw }              ← 非 owner 的最大有效权限
- { tag: OTHER, perms: -- }            ← 其他人权限

default_acl:                                ← 仅目录: 创建子项时复制为子项的 access_acl
- { tag: USER_OBJ, perms: rw }
- { tag: GROUP, subgroup: "editors", perms: rw }
- { tag: GROUP, subgroup: "viewers", perms: r }
- { tag: MASK, perms: rw }
- { tag: OTHER, perms: -- }

subgroups:                                ← 群签名子群定义 (BBS+ with selective
disclosure, 见下方曲线兼容性说明)
- name: "editors"
  members:
    - { pubkey: PK_bob, credential: <Method42 加密> }
    - { pubkey: PK_charlie, credential: <Method42 加密> }
- name: "viewers"
  members:
    - { pubkey: PK_dave, credential: <Method42 加密> }

```

credential 使用 Method 42 (ECDH) 加密存储在链上: `Method42_Encrypt(PK_owner → PK_member, raw_credential)`。每个成员只能解密自己的 credential。

BBS+ 曲线兼容性说明

技术约束: BBS+ 签名方案需要配对友好曲线 (pairing-friendly curves), 如 BLS12-381。secp256k1 不支持双线性配对 (bilinear pairing), 因此无法直接在 secp256k1 上使用 BBS+。

解决方案: 从同一 BIP39 seed 通过独立派生路径生成 BLS12-381 密钥对: - 派生方式: 使用 seed 的 HKDF 派生, info="bitfs-blsl2-381" - 或使用 BIP32 purpose=13 路径: m/13'/236'/0'/... - BLS12-381 密钥仅用于群签名/群加密操作, secp256k1 密钥继续用于 Metanet/ECDH/交易签名

备选方案: 环签名 (Ring Signature) 兼容 secp256k1, 但签名体积更大, 且不支持子群属性 (selective disclosure)。适用于不需要子群细分的简单多用户场景。

ACL 引用方式

ChildEntry 中通过 `acl_ref` 字段指向 ACL 节点的 pubkey (非 txid)，自动解析到最新版本：

```
ChildEntry:
  name:          "shared-doc"
  target_pnode:  PK_node      ← 数据节点的 pubkey
  type:          FILE | DIR | ...
  acl_ref:       PK_acl       ← ACL 节点的 pubkey (永久标识)
```

ACL 共享：多个 ChildEntry 可指向同一个 ACL 节点。更新该 ACL 节点时，所有引用者自动生效：

```
/project/      acl_ref → PK_acl_1
├─ a.txt       acl_ref → PK_acl_1    ← 共享，权限相同
├─ b.txt       acl_ref → PK_acl_1    ← 共享
└─ secret.md   acl_ref → PK_acl_2    ← 独立 ACL，权限不同
```

版本解析：`acl_ref` 使用 pubkey 标识 ACL 节点，自动解析到最新版本（Metanet 中节点用 pubkey 标识，txid 仅是当前版本）。Owner 更新 ACL 节点后，所有引用者无需修改即可看到新权限。

继承机制：创建时复制 (Unix 方式)

参照 Unix POSIX ACL 的 default ACL 机制，采用创建时复制而非运行时继承：

```
在 /project/ 下创建新文件：
→ 新文件的 access_acl = 父目录的 default_acl (复制一份)
→ 复制后独立，修改父目录的 default_acl 不影响已有子文件

在 /project/ 下创建新子目录：
→ 子目录的 access_acl = 父目录的 default_acl
→ 子目录的 default_acl = 父目录的 default_acl (传播)
```

优化：如果子项权限与父项完全一致，`acl_ref` 直接指向同一个 ACL 节点（共享），无需新建。需要独立权限时才创建新 ACL 节点（copy-on-write）。

优势：每个节点自包含，权限检查不需要遍历 Metanet DAG。在区块链文件系统中，避免遍历 DAG 的开销。

权限检查算法

```
check_access(requester_pubkey, node, requested_perm):
```

```

acl = resolve(node.acl_ref)    // 通过 pubkey 解析到 ACL 节点最新版本

// 1. Owner 检查
if requester_pubkey == acl.owner:
    return check(acl.access_acl[USER_OBJ].perms, requested_perm)

// 2. 命名用户检查
for entry in acl.access_acl where tag == USER:
    if entry.pubkey == requester_pubkey:
        return check(entry.perms & acl.mask, requested_perm)

// 3. 群签名子群检查
for entry in acl.access_acl where tag == GROUP:
    if requester has valid credential for entry.subgroup:
        return check(entry.perms & acl.mask, requested_perm)

// 4. Other
return check(acl.access_acl[OTHER].perms, requested_perm)

```

操作流程

写入 (Bob, editor 子群) :

1. 读取: $\text{GroupDecrypt}(\text{Bob 的密钥}, \text{encrypted_K}) \rightarrow K$
 $\text{AES-GCM_decrypt}(K, \text{ciphertext}) \rightarrow \text{plaintext}$
2. 修改内容
3. 加密: $K' = \text{random}()$
 $\text{new_ciphertext} = \text{AES-GCM}(K', \text{new_plaintext})$
 $\text{encrypted_K}' = \text{GroupEncrypt}(\text{GPK}, K')$ ← 子群级别加密
4. 签名: $\sigma = \text{GroupSign}(\text{Bob 的 credential}, \text{transaction})$
 σ 证明: "签名者是 editors 子群的合法成员"
5. 创建 Metanet 节点: $(\text{new_ciphertext}, \text{encrypted_K}', \sigma)$

读取 (Dave, viewer 子群) :

1. 获取节点: $(\text{ciphertext}, \text{encrypted_K})$
2. 解密: $K = \text{GroupDecrypt}(\text{Dave 的密钥}, \text{encrypted_K})$
3. 内容: $\text{plaintext} = \text{AES-GCM_decrypt}(K, \text{ciphertext})$

不需要单独的 key capsule 或 K_{root} 分发。群加密统一处理密钥分发。

创建新文件 (Bob, editor 子群) :

1. 检查 Bob 对父目录的 w 权限 (群签名验证)
2. ACL 处理:
 - a. 新文件与父目录权限相同 → acl_ref 指向同一个 ACL 节点

- b. 需要不同权限 → Owner 创建新 ACL 节点 (copy-on-write)
3. 加密内容: $K = \text{random}()$, $\text{GroupEncrypt}(\text{GPK}, K)$
4. 群签名: $\sigma = \text{GroupSign}(\text{Bob 的 credential}, \text{transaction})$
5. 创建 Metanet 节点 + 更新父目录 ChildEntry

成员管理

群签名的核心优势：成员增减只需 Owner 单独操作，GPK 不变。

添加成员：

1. Owner 为新成员签发 credential (Owner 与新成员的 2-party 协议)
2. Owner 用 Method 42 加密 credential: $\text{Method42_Encrypt}(\text{PK_owner} \rightarrow \text{PK_new}, \text{raw_cred})$
3. Owner 更新 ACL 节点 (添加成员条目 + 加密后的 credential)
4. 完成。现有成员无需参与，GPK 不变。

移除成员：

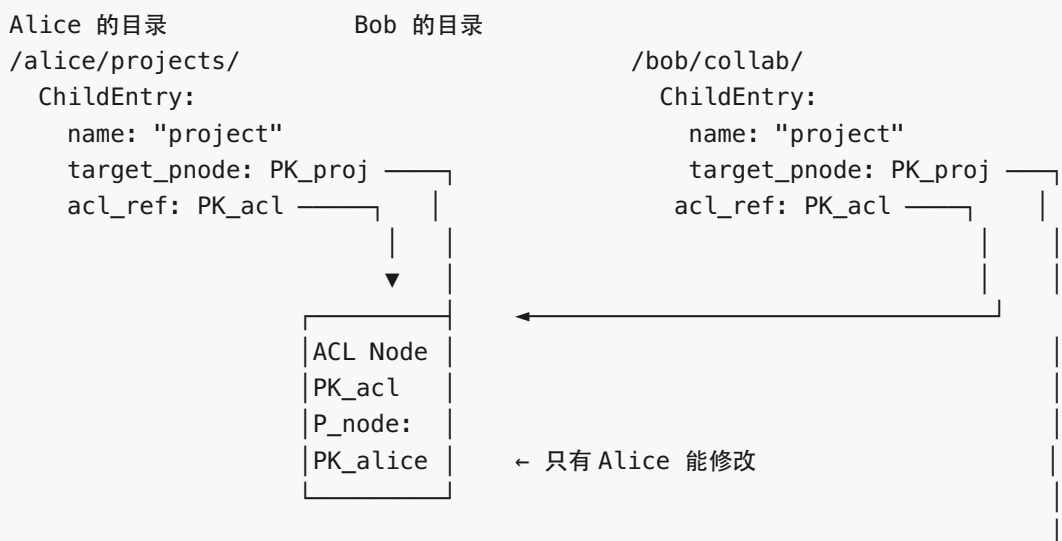
1. Owner 撤销成员的 credential
2. Owner 更新 ACL 节点 (移除成员条目)
3. (可选) Owner 用新 GPK 重新加密内容 → 被移除成员无法解密新版本
4. 完成。现有成员无需参与。

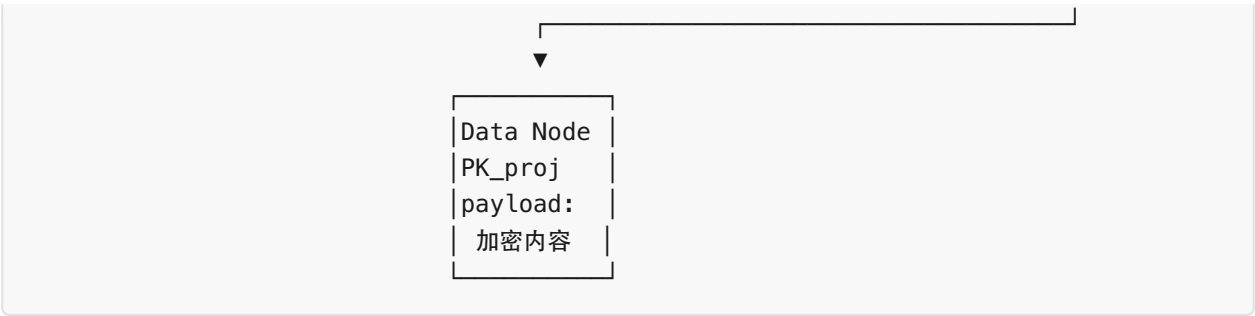
修改权限：

Owner 单独更新 ACL 节点即可。其他成员不参与。

多用户目录结构

每个参与者在自己的目录中创建 ChildEntry 指向共享节点，引用同一个 ACL 节点：





Unix 文件系统映射（扩展）

在第三节的基础上，权限管理增加以下映射：

Unix 概念	BitFS 对应
inode.uid (owner)	ACL 节点中的 owner 字段 (PK_owner)
inode.gid (group)	群签名子群 (GPK + subgroup name)
mode bits (rwxrwxrwx)	ACL 节点中的 access_acl 条目
POSIX ACL (xattr)	ACL 节点（独立 Metanet 节点，通过 acl_ref 引用）
default ACL	ACL 节点中的 default_acl （目录专用，创建时复制）
ACL_MASK	ACL 节点中的 MASK 条目（非 owner 最大有效权限）
chmod/setfacl	Owner 更新 ACL 节点
/etc/group	群签名子群定义（存在 ACL 节点中）

安全特性

特性	说明
匿名写入	群签名保证：验证者知道” 签名者是合法 editors” ，但不知道具体是谁
可追溯	Owner 可打开群签名识别具体签名者，用于审计
密码学读控制	群加密保证：没有 credential 的人数学上无法解密
链上公开存储	credential 用 Method 42 加密存储在链上，只有本人能解密
成员变更无扰	

特性	说明
	增删成员不影响 GPK，现有成员的 credential 仍然有效
自包含权限	创建时复制，每个节点独立验证权限，无需遍历 DAG

与门限签名的对比

群签名方案相比门限签名的优势：

	门限签名 (Threshold)	群签名 (Group)
签名操作	t-of-n 协作，多轮通信	任意成员独立完成
加成员	全员重新 DKG，生成新 GPK	Owner 单独签发 credential，GPK 不变
删成员	全员重新 DKG	Owner 单独撤销 credential
分组	不直接支持	子群 + 属性，天然支持
匿名性	无	验证者不知道谁签的
配套加密	需要额外的 capsule/K_root 机制	群加密统一解决

与现有 Method 42 的关系

群签名/群加密不替代 Method 42，而是在其上层：

- 单用户场景（私有文件、买卖交易）：继续使用 Method 42 ECDH，无需 ACL

- 多用户协作场景：ACL 节点 + 群签名/群加密，credential 通过 Method 42 分发
- ChildEntry 无 acl_ref 时：使用节点 P_node 的 owner 权限（默认行为，向后兼容）

二十三、bsync / bput 一同步与上传

bsync 和 bput 是 b* 工具集中的同步工具，Agent-first 设计（非交互，JSON 输出，可管道组合）。bsync 负责目录级同步，bput 是 bget 的写入对偶。Shell 交互模式中的 put/mput 命令面向人类用户。

rsync → bsync 映射

rsync 概念	bsync 映射	说明
source / destination	local dir / bitfs:// URI	本地目录 ? BitFS 远程
file list (mtime+size)	本地: mtime+hash / 远程: P_node+txid	双方建立文件清单
rolling checksum delta	不需要	BitFS 文件整体加密，无法块级比较
<code>--delete</code>	<code>--delete</code>	同步删除
<code>--dry-run</code>	<code>--dry-run</code>	预览不执行
<code>--checksum</code>	默认行为	BitFS 天然有 SHA-256 hash
单向	双向	bsync 默认双向，可选单向

为什么不需要块级 delta: rsync 的滚动校验和/块级差异是为了减少网络传输。但 BitFS 文件整体 AES-256-GCM 加密——改一个字节整个密文都变——块级 delta 无意义，退化为文件级同步。

bsync 三阶段流水线

参照 rsync 的 Generator → Sender → Receiver 架构：

阶段 1: Scan（文件清单）
└─ 扫描本地目录 → local_list: { path, hash, mtime, size }

└ 解析远程 BitFS → remote_list: { path, pnode, txid, size }

阶段 2: Diff (差异检测)

- └ 加载 sync state (上次同步记录)
- └ 三方比较: local vs remote vs last_sync
- └ 生成 action_list: [PUSH, PULL, DELETE_LOCAL, DELETE_REMOTE, CONFLICT, SKIP]

阶段 3: Apply (执行同步)

- └ PUSH: 加密 + 构造 Metanet 交易 + 广播
- └ PULL: 下载 + 解密 + 写入本地
- └ DELETE_LOCAL: 删除本地文件
- └ DELETE_REMOTE: 删除远程 ChildEntry
- └ CONFLICT: 按策略处理
- └ 更新 sync state

三方差异检测算法

双向同步的核心。需要三方状态：当前本地、当前远程、上次同步快照。

last_sync (快照)
/ \
local (当前) remote (当前)

对于每个路径:

- L = local 相对 last_sync 是否变化
- R = remote 相对 last_sync 是否变化

L R	不变	变了	新增 (不在 sync 中)	已删 (在 sync 中但消失)
不变	SKIP	PULL	—	DELETE_LOCAL
变了	PUSH	CONFLICT	—	PUSH (本地改了远程删了)
新增	—	—	见下	—
已删	DELETE_REMOTE	CONFLICT	—	SKIP (双方都删了)

特殊情况: - 双方都新增 (同名): CONFLICT - 仅本地新增: PUSH - 仅远程新增: PULL

首次同步 (无 sync state): - 文件仅本地存在 → PUSH - 文件仅远程存在 → PULL - 双方都有, 内容相同 → SKIP (记录到 sync state) - 双方都有, 内容不同 → CONFLICT (按策略处理, 默认 skip)

Sync State 文件

~/.bitfs/sync/{sync_id}.json

```
{
  "id": "a1b2c3",
  "local_root": "/home/alice/project/",
  "remote_root": "bitfs://example.com/project/",
  "vault": "personal",
  "last_sync": "2026-02-16T08:00:00Z",
  "files": {
    "docs/readme.md": {
      "local_hash": "sha256:abc123...",
      "local_mtime": 1739600000,
      "local_size": 4096,
      "remote_pnode": "02abc...",
      "remote_txid": "tx_aaa..."
    },
    "src/main.go": {
      "local_hash": "sha256:def456...",
      "local_mtime": 1739600100,
      "local_size": 2048,
      "remote_pnode": "02def...",
      "remote_txid": "tx_bbb..."
    }
  }
}
```

sync_id 由 `SHA256(local_root + remote_root)[:12]` 确定性生成，同一对本地/远程目录总是映射到同一个 sync state 文件。

bsync 命令接口

```
# 双向同步
bsync ./project/ bitfs://example.com/project/

# 预览 (不执行)
bsync --dry-run ./project/ bitfs://example.com/project/

# 单向
bsync --push-only ./project/ bitfs://example.com/project/
bsync --pull-only ./project/ bitfs://example.com/project/

# 冲突策略
bsync --on-conflict=skip ... # 默认: 跳过冲突文件
bsync --on-conflict=ours ... # 本地优先 (总是 push)
bsync --on-conflict=theirs ... # 远程优先 (总是 pull)
bsync --on-conflict=fail ... # 有冲突立即退出

# 删除同步
bsync --delete ./project/ bitfs://example.com/project/
```

```
# 输出格式 (Agent-first)
bsync --output json ./project/ bitfs://example.com/project/
```

JSON 输出示例:

```
{
  "actions": [
    {"path": "docs/new.md", "action": "PUSH", "reason": "local_new"},
    {"path": "src/updated.go", "action": "PULL", "reason": "remote_changed"},
    {"path": "old/removed.md", "action": "DELETE_LOCAL", "reason": "remote_deleted"},
    {"path": "both/changed.md", "action": "SKIP", "reason": "conflict"}
  ],
  "summary": {
    "push": 1, "pull": 1, "delete_local": 0, "delete_remote": 1,
    "conflict": 1, "skip": 42, "total": 46
  }
}
```

bput 设计

bget 的写入对偶。本质是 `bsync --push-only` 的简化单文件/单目录版本，不维护 `sync state`。

```
# 上传单文件
bput ./readme.md bitfs://example.com/docs/readme.md

# 上传目录 (递归)
bput -r ./src/ bitfs://example.com/project/src/

# 更新已有文件 (同 P_node, 新 txid)
bput ./readme.md bitfs://example.com/docs/readme.md

# 输出格式
bput --output json ./file bitfs://example.com/file
```

bput 不接受 `--public` / `--private` / `--paid` 参数。加密级别由文件系统现有设置决定 (ACL 或默认 owner 权限)，上传与加密策略分离。

bput vs bitfs put: - `bput` 一 Agent-first, 非交互, JSON 输出, 可管道 - `bitfs put` / `shell put` 一 面向人类, 可交互, 支持进度条

b* 工具集总览 (更新)

只读工具:	读写工具:	
bls	bput	← 新增 (上传)

bcat	bsync	← 新增（同步）
bget		
bstat		
btree		
